

Emil Johnsen
Zahid André Kristiansen
Svein Kåre Sørestad

Gaussian-Driven Model-Based Reinforcement Learning with Vector Quantisation for State Generalisation

Bachelor ingeniørfag, data Bachelor-oppgave
Veileder: Ole Christian Eidheim
May, 2025

Norges teknisk-naturvitenskapelige universitet
Fakultet for informasjonsteknologi og elektronikk
Institutt for datateknologi og informatikk

Abstract

Humans can learn new tasks with remarkably few interactions, often forming generalised concepts after only a handful of experiences. In contrast, most reinforcement learning (RL) algorithms, particularly model-free methods, require hundreds of thousands of environment steps to achieve competent performance. This study investigates whether incorporating biologically inspired mechanisms can improve sample efficiency in model-based RL. Specifically, it examines how vector quantisation of encountered states influences the performance of a model-based agent. Inspired by how the brain appears to compress and organise information post-experience, clustering is used here as a means of abstraction and generalisation. A custom model-based agent was developed and evaluated in the Cart-Pole environment, with key experiments exploring the effect of the Gaussian width σ in the agent's reward approximation mechanism. Results show that without clustering, smaller σ values yield better performance; however, when clustering is applied, larger σ values lead to significant improvements. Impressively, experiments demonstrate that for specific values of the Gaussian width, the clustered agent is able to outperform the pre-clustered agent - even when the state space is compressed to as few as 500 representative states. These findings suggest a strong interaction between representation granularity and abstraction mechanisms, offering insight into sample-efficient learning in data-scarce environments. ¹

¹The source code utilised in this study is publicly available at: <https://github.com/EmilJohns1/BachelorThesis>. Additionally, all raw experimental data can be accessed at: <https://github.com/SveinKare/BachelorThesis-logs>.

Sammendrag

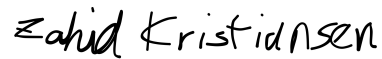
Mennesker kan lære nye oppgaver med bemerkelsesverdig få interaksjoner, og danner ofte generaliserte konsepter etter bare noen få erfaringer. I kontrast krever de fleste algoritmer i forsterkende læring, spesielt modellfrie metoder, hundretusener av interaksjoner med miljøet for å oppnå kompetent ytelse. Denne studien undersøker om biologisk inspirerte mekanismer kan forbedre samplingeffektiviteten i modellbasert RL. Spesifikt ser den på hvordan vektor-kvantisering av erfarte tilstander påvirker ytelsen til en modellbasert agent. Inspirert av hvordan hjernen ser ut til å komprimere og organisere informasjon etter erfaring, brukes klyngeanalyse her som et middel for abstraksjon og generalisering. En tilpasset modellbasert agent ble utviklet og evaluert i Cart-Pole-miljøet, med sentrale eksperimenter som utforsker effekten av Gaussisk bredde σ når agenten approksimerer gevinst. Resultatene viser at uten klynging gir mindre σ bedre ytelse; men med klynging fører større σ til betydelige forbedringer. Imponerende nok viser eksperimentene at for bestemte verdier av Gaussisk bredde kan agenten etter klynging overgå ytelsen den hadde før klynging - selv når tilstandsrommet komprimeres til så få som 500 representative tilstander. Disse funnene tyder på en sterk samhandling mellom representasjonsgranularitet og abstraksjonsmekanismer, og gir innsikt i sampleeffektiv læring i miljøer med lite data.

Preface

We chose this assignment as it represented an opportunity to contribute to an exciting field of research. All the members of the research team were passionate about the field of machine learning, so a project of this nature proved to be a challenging and educational experience. We would like to thank our supervisor, Ole Christian Eidheim, who proposed the assignment and helped the group greatly both with his knowledge and enthusiasm. We would also like to thank our families for their support throughout our years of study.



Emil Johnsen



Zahid André Kristiansen



Svein Kåre Sørstad

Research question

The purpose of this study was to implement and test a biologically plausible reinforcement learning agent. Using this agent as a basis, different methods of generalising states were tested. Primarily, these methods were intended to produce an agent that could be trained to a satisfactory degree with fewer episodes.

Contents

Abstract	i
Sammendrag	ii
Preface	iii
Research question	iv
Contents	v
Figures	viii
Tables	x
Acronyms	xi
1 Introduction	1
1.1 Research Question	1
1.2 Thesis Structure	1
2 Theory	3
2.1 Reinforcement Learning	3
2.1.1 Key Principles	3
2.1.2 Environments	4
2.1.3 Markov Decision Process	5
2.1.4 Bellman Equations	6
2.1.5 Model-Based Reinforcement Learning	7
2.1.6 Applications of Model-Based RL	7
2.1.7 Model-Free Reinforcement Learning	8
2.1.8 Applications of Model-Free RL	9
2.2 Statistical methods	10
2.2.1 The Softmax Function	10
2.2.2 Gaussian functions	11
2.2.3 Linear Regression	11
2.2.4 Splines	11
2.3 Vector quantisation	12
2.3.1 K-means Clustering	12
2.3.2 Gaussian Mixture	12
2.3.3 Online Arbitrary Shaped Clustering	13
2.4 Dimensionality Reduction	13

2.4.1 t-SNE	13
2.5 Numerical Instability with Floating Point Numbers	15
2.6 Frameworks	15
2.6.1 Gymnasium	15
2.6.2 Cart-Pole	15
2.6.3 Observation- and Action Space for Cart-Pole	16
2.6.4 Lunar Lander	16
2.6.5 Observation- and Action Space for Lunar Lander	17
2.6.6 Reward Functions	18
2.7 Biological plausibility	18
2.8 Sustainability	19
2.8.1 Goals and Frameworks	19
3 Related Work	20
3.1 Reinforcement Learning in Low-Dimensional Environments	20
3.2 Sample Efficiency in Reinforcement Learning	21
4 Method	26
4.1 Choice of Technology and Development methods	26
4.1.1 Programming Language	26
4.1.2 Libraries	26
4.1.3 Environments	27
4.1.4 Research Method and Collaborative Framework	27
4.2 Agent Design	28
4.2.1 Model	29
4.2.2 Policy	29
4.2.3 Transition Model	30
4.2.4 Clustering	30
4.2.5 Model-Free Comparisons	31
4.3 Experimental Design	32
4.3.1 Experiments	32
4.4 Sustainability Assessments	34
5 Results	35
5.1 Direct Transition Model without Clustering	35
5.2 Direct Transition Model with Regular K-Means	36
5.3 Weighted K-Means	39
5.4 Direct Transition Model with Alternative Clustering Techniques	42
5.4.1 Online Clustering	42
5.4.2 Gaussian Mixture Model	43
5.5 Delta Transition Model using Splines	44
5.6 Delta Transition Model using Linear Regression	47
5.7 Weighted Clustering with Higher Gaussian Widths	49
5.7.1 Delta Transition Model	49

5.7.2 Direct Transition Model	51
5.8 Neuron Activation States	53
5.8.1 Gaussian Function Approach	53
5.8.2 Positional Encoding	54
5.9 DQN and Q-Learning	55
6 Discussion	58
6.1 Direct Transition Model without Clustering	58
6.2 Direct Transition Model with Regular K-Means	58
6.3 Weighted K-Means	59
6.4 Direct Transition Model using Online Clustering	60
6.4.1 Online Clustering	60
6.4.2 Gaussian Mixture Model	61
6.5 Delta Transition Model using Splines	61
6.6 Delta Transition Model using Linear Regression	62
6.7 Weighted Clustering with Higher Gaussian Widths	63
6.7.1 Delta Transition Model	63
6.7.2 Direct Transition Model	64
6.7.3 Summary	64
6.8 Neuron Activation States	64
6.9 DQN and Q-Learning	65
7 Conclusion and Future Work	67
7.1 Findings	67
7.2 Impact	68
7.3 Future work	68
Bibliography	70
Appendix A Hyperparameters	i
Appendix B Q-Learning Reproduction Attempt	vii
Appendix C Raw data	ix

Figures

2.1	Comparison of SNE and t-SNE on MNIST2500	14
2.2	An illustration of the Cart-Pole environment [80].	16
2.3	A visualisation of the Lunar Lander environment [8].	17
5.1	Running mean and standard deviation for the different variations of the experiment	35
5.2	t-SNE plot of the agent's states after training	36
5.3	Results from the second experiment, with separate lines for each variation	37
5.4	Zoomed in view of figure 5.3	38
5.5	Comparison of experiments I and II	38
5.6	Direct transition model with exponential weighting	39
5.7	Direct transition model with softmax weighting	40
5.8	Closeup of figure 5.7	41
5.9	Log-softmax weighting	41
5.10	Comparison of experiments I, II and III	42
5.11	Variations of the experiment with online clustering. Note that the scale here is different due to the lower overall values. . .	43
5.12	Running mean reward for Gaussian mixture model experiment	44
5.13	Comparison of different Gaussian widths using splines for the delta transition model, without applying clustering.	45
5.14	Comparison of different error thresholds for the Mean Squared Error (ϵ)	46
5.15	Comparison of different k values for the Delta Variant	47
5.16	Comparison of different Gaussian widths using linear regression for the delta transition model, without applying clustering.	48
5.17	Weighted k-means clustering using $\sigma = 0.5$	48
5.18	Post-clustering results for Gaussian width 1.0	49
5.19	Post-clustering results for Gaussian width 2.5	50
5.20	Post-clustering results for Gaussian width 5.0	50
5.21	Post-clustering results for Gaussian width 1.0	51
5.22	Post-clustering results for Gaussian width 2.5	52
5.23	Post-clustering results for Gaussian width 5.0	52
5.24	Representation of states through neuron activation using a Gaussian function.	53
5.25	Regular positional encoding with <i>sin</i> and <i>cos</i> encodings.	54
5.26	Positional encoding with cut-off for negative values.	55
5.27	Mean episode reward during training	56

5.28	Running mean reward during testing	57
B.1	Mean episode reward over 100 runs in the Q-learning reproduction attempt.	vii

Tables

2.1	Observation space for the Cart-Pole environment [16].	16
2.2	Observation space for the Lunar Lander environment [17]. . .	17
A.1	Hyperparameters used for experiment I, Figure 5.1.	i
A.2	Hyperparameters used for experiment II, Figure 5.3.	i
A.3	Hyperparameters used in experiment III, Figure 5.6.	i
A.4	Hyperparameters used in experiment III, Figure 5.7.	i
A.5	Hyperparameters used in experiment III, Figure 5.9.	ii
A.6	Hyperparameters used in experiment IV, Figure 5.11.	ii
A.7	Hyperparameters used in experiment IV, Figure 5.12.	ii
A.8	Hyperparameters for experiment V, Figure 5.13.	ii
A.9	Hyperparameters for experiment V, Figure 5.14.	iii
A.10	Hyperparameters for experiment V, Figure 5.15.	iii
A.11	Hyperparameters for experiment VI, Figure 5.16.	iii
A.12	Hyperparameters for experiment IV, Figure 5.17.	iii
A.13	Hyperparameters for experiment VII, Figure 5.18.	iv
A.14	Hyperparameters for experiment VII, Figure 5.19.	iv
A.15	Hyperparameters for experiment VII, Figure 5.20.	iv
A.16	Hyperparameters for experiment VII, Figure 5.21.	iv
A.17	Hyperparameters for experiment VII, Figure 5.22.	iv
A.18	Hyperparameters for experiment VII, Figure 5.23.	v
A.19	Hyperparameters for experiment VIII, Figure 5.24.	v
A.20	Hyperparameters for experiment VIII, Figure 5.25.	v
A.21	Hyperparameters for experiment VIII, Figure 5.26.	v
A.22	Hyperparameters for the DQN agent experiment IX (training), 5.27.	vi
A.23	Hyperparameters for the DQN agent experiment IX (testing), 5.28.	vi
A.24	Hyperparameters for the Q-learning baseline (training), ?? .	vi
A.25	Hyperparameters for the Q-learning baseline (testing), ?? .	vi
B.1	Hyperparameters used in the Q-learning reproduction attempt	viii

Acronyms

AI	Artificial Intelligence.
DDPG	Deep Deterministic Policy Gradient.
DQN	Deep Q-network.
MBRL	Model-Based Reinforcement Learning.
MDP	Markov Decision Process.
PPO	Proximal Policy Optimization.
RL	Reinforcement Learning.
SAC	Soft Actor-Critic.
t-SNE	t-distributed Stochastic Neighbor Embedding.

1. Introduction

As the field of artificial intelligence progresses, there is an increasing desire to relate these new revolutionary methods to our understanding of human intelligence. Biological plausibility refers to methods that can be linked, often directly, to our current understanding of the brain. In this study, the process of sleeping served as the primary source of inspiration for the methods implemented. This process was imitated by clustering a model-based agent's learned states and attempting to generalise the data to improve its performance.

One of the biggest challenges in model-based reinforcement learning is the considerable amount of necessary calculations performed as the agent gathers data. Unlike its model-free counterparts, the model-based agent builds a model of the environment using the data it collects. Subsequently, querying this model for its decision-making causes an increasingly large computational overhead.

In the same way that the human brain organises experiences during sleep, clustering algorithms could improve the agent's sample efficiency by generalising data. Additionally, compressing the agent's model without losing performance could substantially reduce the computational overhead.

1.1 Research Question

This study investigated the impact of using vector quantisation techniques on a model-based reinforcement learning agent. The goal was to improve the performance of the agent through these methods, enabling sufficient agents to be trained with fewer episodes.

1.2 Thesis Structure

The following chapter of the report will cover the necessary theory to gain a deeper understanding of the subject. The theory chapter will primarily focus on machine learning, with some relevant topics from statistics. Additionally, this chapter will cover the frameworks used and the sustainability concepts relevant to the study.

Following this, the method section will describe the agent that was designed and implemented. This section will go into detail on the agent's policy and model to allow reproduction of the results. Additionally, this

section will describe the experiments performed to test this agent.

Next, the results of the experiments mentioned above will be presented, including graphs and plots to visualise the results. The results are organised by experiment in the same order as they are listed in the method section.

Afterwards, the results will be discussed and related to the research question. In this chapter, the results will be analysed, compared, and related to the research question.

Finally, the report will attempt to reach a conclusion based on the results and discussion. Additionally, the impact of the study will be considered with respect to the environmental concerns covered in the theory and method section. Finally, future work for the study is covered to conclude the report.

2. Theory

This chapter details the relevant theory needed to fully understand the contents of the study. It covers essential aspects of reinforcement learning (RL), including core principles and statistical methods used for model-based approaches. To contextualise the experimental setup, an overview of the relevant frameworks is presented. Finally, the sustainability of Artificial Intelligence (AI) is briefly discussed, highlighting the importance of developing energy-efficient algorithms.

2.1 Reinforcement Learning

In this section, the fundamental concepts of RL are covered. It will outline the basic principles that define the RL field and provide an overview of how these principles are implemented in practical applications. Rather than offering a comprehensive overview of the field, this section focuses on concepts most relevant to this study.

2.1.1 Key Principles

RL is a subfield of machine learning where a computer program, referred to as an agent, interacts with an environment through actions and receives rewards as feedback. Rather than learning from labelled data, as is the case in supervised learning, it must explore the environment and learn new behaviours. RL is commonly modelled as a set of states $s \in \mathcal{S}$, a set of actions $a \in \mathcal{A}$, and a set of scalar rewards $r \in \mathcal{R}$. The agent receives the current state, for example in the form of a vector. Given this state, an action is chosen that causes a change in the agent's state, and the value of this change is received by the agent as a reward in the form of a scalar value. The objective in RL is for the agent to learn and optimise a strategy to maximise the rewards it receives. This strategy is referred to as a policy π and is often defined as a function that takes a state as input and outputs an action [39], [58].

$$\pi(s) = a \tag{2.1}$$

$$\pi(a \mid s) \tag{2.2}$$

Policies are divided into two groups: deterministic and stochastic policies. Equation 2.1 describes a deterministic policy, where the agent will always select an action a given a state s . In this case, there is no randomness involved in the agent's decision-making, so the policy functions as a dir-

ect mapping between states and actions. Stochastic policies, on the other hand, shown in equation 2.2, provide a probability distribution of the possible actions. Instead of the output being an action, stochastic policies output a probability for a given action, which is calculated based on its estimated utility. The agent uses this distribution to select its moves, which also enables a higher degree of exploration due to the fact that low-probability actions still have a chance of being chosen [22], [39].

As mentioned above, one of the key traits of RL is the fact that the agent must explore its environment. A fundamental problem that arises is the trade-off between exploration and exploitation. On the one hand, the agent should optimise its rewards by exploiting strategies that have proven to work; on the other hand, there may be other possible strategies that yield higher rewards that have not yet been discovered. Achieving a satisfactory policy usually requires some way of balancing the agent's exploration and exploitation. A simple RL problem that illustrates this issue is the K-armed bandit problem. The agent is placed in a room with k gambling machines that can be pulled, and a reward is paid out based on some probability that differs between the machines. Any of the machines can be pulled at any time, and they do not require a deposit. Since the agent does not know the probabilities of the machines, it must explore and experiment to estimate the value of each machine [22], [39].

2.1.2 Environments

Environments used for reinforcement learning can have many properties that describe how the environment behaves and how the agent can interact with it. This section will cover some fundamental properties, specifically those pertinent to the problem domain.

As previously stated in Section 2.1.1, an RL agent receives some description of its current state from the environment, which it uses to choose an action. In certain environments, the agent can observe every relevant detail. This is referred to as a fully observable environment. However, in the real world, one often encounters issues like noise and imprecise sensors, which may lead to unreliable interpretations of the current state. These environments are called partially observable. If the agent cannot observe anything at all about its current state, the environment is unobservable [58].

Furthermore, the available actions, as well as the states, can be either continuous or discrete. Although both states and actions can be discrete or continuous, they do not need to be the same. An environment can have continuous states and discrete actions or vice versa. Additionally, there can be an element of randomness in the environment. An environment where taking the same action in a given state always gives the same res-

ult is called deterministic; otherwise, it is considered stochastic. Finally, an environment can be considered static or dynamic. A static environment is one where changes can only happen as a result of the agent's actions. Dynamic environments can change while the agent is choosing an action, meaning that the computing speed of the agent plays a role in its performance [58].

Time-steps

A time step is the unit of time in reinforcement learning. Representing a single interaction between the agent and its environment. At each discrete time step, the agent observes the current state, chooses an action, and, in response, receives some immediate reward. Time is divided into these successive steps, during which the agent makes decisions and learns from feedback.

Episodes

In reinforcement learning, the interaction between the agent and the environment is naturally organised into episodes. An episode consists of a sequence of states, actions, and rewards. Starting from an initial state, it ends when a terminal or predefined condition is met. An example of a terminal state would be a loss in a video game, but one could predefine a condition by the number of points. After an episode, the environment often resets to an initial state again.

Runs

During training, an agent will run through many episodes to train its policy. A run refers to one independent execution of a complete learning process or experiment from start to finish. In reinforcement learning theory, a run encompasses all time steps and possibly several episodes required for an agent to either finish training or fulfil a predetermined length.

Time steps partition the interaction into atomic decision points, and episodes group these interactions into meaningful data with a beginning and end, and they run and encapsulate entire experiments. As an earlier example with the K-armed bandits, each attempt to select an arm and receive a reward is a step, and a sequence of arm selection forms an episode. Multiple episodes allow the agent to better estimate the value of each arm by balancing exploration and exploitation.

2.1.3 Markov Decision Process

A *Markov Decision Process* (MDP) is a mathematical framework for modelling sequential decision-making processes under uncertainty. An MDP is

often defined as a four-tuple, varying by the context in which the framework is used. The notation used varies, but the standard definition is as follows [6], [22], [39], [58]:

$$\langle S, A, P, R \rangle,$$

- S is a set of possible states in the system.
- A is a set of actions from which the decision-maker can choose.
- $P(s' \mid s, a)$ is the transition probability, indicating the probability of reaching state s' when action a is taken in the state s . If the state transitions are independent of previous history, we say that the model has the Markov property or that it is a Markovian model.
- $R(s, a)$ is the reward function that assigns an immediate reward for taking action a in state s . Alternatively, the reward can be calculated based on the state s alone.

In some cases, a discount factor γ is also included in the model. A discount factor is a number between 0 and 1 that describes how much weight is put on future rewards when making a value judgement. In the most basic form, the discount factor is raised to the power of the time-step t and multiplied by the expected reward in that time-step. A value of 1 will, in other words, weigh rewards equally no matter how far into the future they are expected to be received [58].

Although the transition function $P(s' \mid s, a)$ is defined as a probability of reaching a state s' , meaning a stochastic environment, it is not uncommon to apply the MDP model to deterministic environments as well. In such cases, the probability of a transition is always 1, $P(s' \mid s, a) = 1.0$, meaning the MDP can be described as deterministic [6].

MDPs are often used in areas such as artificial intelligence, robotics, and operations research to find optimal strategies in complex and uncertain environments. By modelling a decision problem as an MDP, policy optimisation can be defined clearly and with standardised notation.

Optimisation in MDPs can be carried out using dynamic programming, linear programming, or iterative methods such as value iteration and policy iteration. These methods aim to find an optimal value or action function that maximises the cumulative reward over time.

2.1.4 Bellman Equations

Using the framework of the MDP, the agent's goal of optimising a policy can be mathematically defined. The policy, as stated in Section 2.1.1, is a strategy the agent develops to achieve optimal rewards. However, future rewards are hard to predict even in deterministic environments due to

the sheer amount of calculations that are needed for longer horizons. To simplify the problem, it can be useful to instead designate a utility of a given state. The utility of a state can be given as:

$$U(s) = R(s) + \gamma \max_{a \in A(s)} \sum_{s'} P(s' | s, a) U(s') \quad (2.3)$$

The above equation is known as a Bellman equation. In this equation, the utility of a state is defined as the immediate reward in the state plus the discounted utility of the next state. The action a in the equation is the action that optimises future utility. Bellman equations are integral to optimisation problems in dynamic programming and are often used as a basis for policy optimisation [4], [22], [58].

2.1.5 Model-Based Reinforcement Learning

Model-based reinforcement learning methods include an explicit model of the environment's dynamics. In this context, a model refers to a representation of how an environment behaves, typically the transition function (which predicts the next state given a current state and action) and the reward function (predicting the immediate reward for a given action in a state) [32]. By learning or leveraging such a model, an agent can simulate state transitions and outcomes internally without having to execute each action in the real environment [22], [39], [58].

The advantage of model-based approaches is improved sample efficiency. By using the internal model for simulation, the agent can explore multiple future trajectories and evaluate potential decisions without executing them in the actual environment. This accelerates learning by allowing rapid policy improvement using simulated experiences, which is particularly valuable when real-world interactions are costly or limited [34], [58].

2.1.6 Applications of Model-Based RL

Model-Based Reinforcement Learning (MBRL) leverages predictive models of the environment to enhance decision-making and learning efficiency. This section explores notable real-world applications of MBRL across various domains.

Robotics and Control Systems

MBRL has been instrumental in advancing robotic capabilities, particularly in tasks requiring precision and adaptability. For instance, researchers have demonstrated the effectiveness of MBRL on the Boston Dynamics Spot robot equipped with a manipulator, achieving accurate dynamic end-effector trajectory tracking even in low-data regimes. Additionally, Boston

Dynamics has integrated reinforcement learning into Spot’s locomotion control system to handle real-world variability, enabling the robot to traverse challenging terrains [34] [7].

Autonomous Driving

In the realm of autonomous vehicles, MBRL contributes to developing systems that can predict and adapt to dynamic driving conditions. For example, risk-aware approaches have been introduced that combine predictive models with reinforcement learning to enhance safety and decision-making [78] [9].

Gaming

In the area of games like Chess, Go, and Atari, recent algorithms such as MuZero, EfficientZero, and BBF have significantly improved the performance of game-playing agents. These algorithms have combined model-free and model-based techniques to enhance decision-making in complex environments. MuZero, for example, excels in planning without a predefined model, while EfficientZero and BBF focus on improving training efficiency. A more detailed discussion of these works can be found in Section 3.2.

2.1.7 Model-Free Reinforcement Learning

Model-free RL differs from model-based RL by relying on trial-and-error interactions with the environment to learn optimal policies without constructing an explicit model of the environment’s dynamics [22]. This approach allows agents to make decisions based solely on received rewards and experienced transitions rather than predictions of future states and rewards.

Off-Policy and On-Policy

Off-policy methods refer to machine learning methods where the agent learns from a different policy than the one it’s trying to optimise. The agent uses a separate policy, a behaviour policy, during training, which leans more towards exploration to explore its environment. During this phase, the agent uses the knowledge gained to train a target policy, which is then used during the test phase to optimise exploitation [22].

On-policy methods, on the other hand, do not distinguish between the behaviour policy and the target policy. Instead, the policy is continually updated with new information as the agent explores and is purely based on the actions and rewards of the agent [22].

Positional encoding

Positional encoding is a method of incorporating information about the position of data in a sequence within the data itself. A common implementation of positional encoding involves using sine and cosine functions of even and odd dimensions, respectively. Although the information about the position is unique, it only provides a hint about the data's relative position in the sequence. The goal of positional encoding is to allow parallel processing of data where the order matters and thereby make learning faster [21], [71].

Common Model-Free Algorithms

There are various approaches to model-free reinforcement learning, each differing in how they represent and optimise decision-making policies. Some methods focus on estimating value functions, while others directly learn optimal policies through gradient-based updates. Below are some of the most widely used model-free RL algorithms:

Q-Learning

An off-policy algorithm that learns the value of actions in given states, aiming to find the optimal action-selection policy by updating value estimates based on the maximum expected future rewards [10, p. 6], [39, p. 253], [44], [45].

Deep Q-Network (DQN)

An extension of Q-learning that utilises deep neural networks to approximate the Q-value function. DQN incorporates experience replay and target networks to stabilise training, making it suitable for high-dimensional state spaces [51], [50], [45].

Policy Gradient Methods

Algorithms that adjust the policy directly by computing gradients of expected rewards concerning policy parameters are often used in environments with high-dimensional or continuous action spaces. Popular examples of such methods are: Proximal Policy Optimization (PPO), Soft Actor-Critic (SAC) and Deep Deterministic Policy Gradient (DDPG) [35], [26], [36], [27], [62].

2.1.8 Applications of Model-Free RL

Model-free RL algorithms have been successfully applied in various domains. This section explores notable applications in robotics, gaming, and finance.

Robotics

In robotics, model-free RL enables agents to learn complex motor skills and adapt to dynamic environments without explicit models of the system's dynamics. For example, Haarnoja et al. [26] introduced the Soft Actor-Critic (SAC) algorithm, which has been used to achieve robust locomotion in robotic systems. Additionally, Steckelmacher [66] demonstrated the application of model-free RL in real-world robots, highlighting its practicality in the development of adaptive behaviours.

Gaming

Model-free RL has revolutionised the gaming industry by enabling agents to achieve superhuman performance in complex games. Mnih et al. [50] developed a Deep Q-Network (DQN) that learned to play Atari 2600 games directly from raw pixels, surpassing human performance in several instances. Furthermore, Vinyals et al. [72] applied model-free RL to master the real-time strategy game StarCraft II, demonstrating the algorithm's capability to handle high-dimensional state and action spaces.

Finance

In the financial sector, model-free RL is utilised for portfolio optimisation and trading strategies. Sato [60] explored the application of model-free RL in financial portfolios, identifying key challenges and proposing solutions for effective implementation. Moreover, Wang et al. [75] investigated the performance of deep RL agents in asset allocation, showcasing the potential of these algorithms to enhance investment decision-making processes.

2.2 Statistical methods

This section will cover a range of statistical methods relevant to this study. It will discuss the softmax function, which is widely used in machine learning, as well as Gaussian functions, linear regression, and splines.

2.2.1 The Softmax Function

The softmax function maps an n -dimensional vector of real values to a probability distribution of n possible outcomes. All the values are normalised, and the sum of all the values is 1. This function is often used in machine learning, particularly as the last activation function of neural networks. The softmax function is defined as follows [23]:

$$\text{softmax}(\mathbf{x})_i = \frac{\exp(x_i/T)}{\sum_{j=1}^n \exp(x_j/T)}.$$

Alternatively, the logarithm of the function can be used to avoid numerical instability from very small values. This can be useful when the vector has many dimensions, and the difference in values is large [41].

The temperature parameter T is used to control the sharpness of the output distribution. A higher value of T results in a softer, more uniform probability distribution, while a lower value of T makes the distribution more peaked, assigning higher probability to the largest components of the input vector $\mathbf{x}$. Formally, when $T \rightarrow 0$, the softmax output approaches a one-hot vector that assigns probability 1 to the index of the maximum x_i , and when $T \rightarrow \infty$, the distribution approaches a uniform distribution over all n elements:

$$\lim_{T \rightarrow 0} \text{softmax}(\mathbf{x})_i = \begin{cases} 1 & \text{if } i = \arg \max_j x_j \\ 0 & \text{otherwise} \end{cases}, \quad \lim_{T \rightarrow \infty} \text{softmax}(\mathbf{x})_i = \frac{1}{n}.$$

2.2.2 Gaussian functions

Gaussian functions are functions on the base form:

$$f(x) = \exp(-x^2) \quad (2.4)$$

These functions are widely used in statistics for different purposes. One practical application is calculating a normalised exponential measure of distance, where a query point x is compared to a centre μ . This variation can be described as follows:

$$f(\vec{x}) = \exp\left(-\frac{\|(\vec{x} - \vec{\mu})\|_2^2}{\sigma}\right) \quad (2.5)$$

In equation 2.5, σ is a constant that controls the width of the curve of the function and is in some cases referred to as the Gaussian width [25].

2.2.3 Linear Regression

Linear regression is a method to model the relationship between a dependent variable and one or more independent variables by fitting a linear equation to observed data. These models are often fitted by minimising some measure of error, for instance, using the least squares approach [53].

2.2.4 Splines

Splines are functions defined by joining several low-degree polynomials over a given interval. Doing this also requires end conditions to define the behaviour of the function near the endpoints. A common type of spline is

the natural cubic spline, which refers to a group of splines consisting of third-degree polynomials whose second derivatives at the endpoints are zero. [18].

2.3 Vector quantisation

Vector quantisation techniques are used to compress a set of vectors into a smaller set of representative centroids. There are different ways of identifying these centroids. This section will present methods that focus primarily on clustering techniques [24].

2.3.1 K-means Clustering

k -means clustering is an unsupervised learning method for grouping data into k clusters. The method works as follows:

1. Initialize k centroids randomly.
2. Assign each data point to the nearest centroid.
3. Update the centroids by calculating the average of all points in each cluster.
4. Repeat steps 2 and 3 until convergence, that is, until the cluster assignment does not change significantly.

k -means clustering is widely used in data analysis to identify underlying patterns and structures in large datasets.

K-means clustering is an unsupervised learning method that partitions the dataset into k clusters based on the similarity between data points. In this model, k -means is used to group similar states, which reduces the dimensionality of the state space and facilitates the generalisation of both transitions and rewards. By aggregating states, the complexity of the environment model is reduced, making it possible to compute more robust estimates for both transition probabilities and reward functions [1].

2.3.2 Gaussian Mixture

Gaussian Mixture Models are probabilistic models that represent a probability distribution as a combination of multiple Gaussian distributions [52]. Each component in the mixture corresponds to a cluster characterised by its own mean and covariance. This approach allows for modelling data distributions that may not be well-represented by a single Gaussian distribution, accommodating varying shapes, sizes, and orientations of clusters.

Mathematically, the probability density function of GMM is presented as:

$$p(\mathbf{x}) = \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \quad (2.6)$$

where $\mathbf{x} \in R^d$ is a data point, K is the number of mixture components, π_k is the mixing coefficient for component k such that $\sum_{k=1}^K \pi_k = 1$, and $\mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$ is the multivariate normal (Gaussian) distribution with mean $\boldsymbol{\mu}_k$ and covariance matrix $\boldsymbol{\Sigma}_k$.

2.3.3 Online Arbitrary Shaped Clustering

Online arbitrary-shaped clustering is an online clustering algorithm based on a set of randomly generated points which are adjusted and shaped into clusters. This is done by using Gaussian functions f_i to evaluate the impact a given input x should have on centroid μ , as well as accounting for neighbouring influence. This is done by using the following equation:

$$\Delta\mu_i = \frac{\eta}{\sigma} \left(f_i(\mathbf{x})(\mathbf{x} - \boldsymbol{\mu}_i) - 2\lambda \sum_{j \neq i} f_i(\boldsymbol{\mu}_j)(\boldsymbol{\mu}_j - \boldsymbol{\mu}_i) \right) \quad (2.7)$$

where η is the learning rate, and σ and λ are used to adjust the influence of the Gaussian functions [15].

2.4 Dimensionality Reduction

Dimensionality reduction refers to the process of transforming high-dimensional data into a lower-dimensional representation while preserving as much relevant information as possible. This section outlines the dimensionality reduction techniques employed in this study, with a particular focus on methods used to visualise states and their associated rewards in a more interpretable form.

2.4.1 t-SNE

t-Distributed Stochastic Neighbour Embedding (t-SNE) is a dimensionality reduction technique primarily used for visualising high-dimensional data in two or three dimensions. It was introduced by van der Maaten and Hinton [48] as an improvement over Stochastic Neighbour Embedding (SNE), which suffered from the "crowding problem", where the low-dimensional representation could not accurately reflect the high-dimensional distances due to the limited space available [33].

The core idea of t-SNE is to transform the pairwise similarities between data points into probability distributions. In the high-dimensional space,

similarities between data points are represented using Gaussian distributions, while in the low-dimensional space, t-distributions with one degree of freedom are used to capture pairwise similarities. This choice of distribution helps to prevent the crowding problem, as it has heavier tails that are better suited to modelling the global structure of the data [48].

The optimisation of the t-SNE algorithm is based on minimising the *Kullback-Leibler* (KL) divergence between the probability distributions of the high-dimensional and low-dimensional representations. This divergence quantifies the difference between the two distributions, with the goal being to make the low-dimensional representation as similar as possible to the high-dimensional one [48].

One of the key parameters in t-SNE is *perplexity*, which controls the balance between the local and global structure of the data. A higher perplexity considers a larger number of nearest neighbours, which focuses more on global relationships, while a smaller perplexity emphasises local structure [48].

While t-SNE excels at preserving local neighbourhoods, it does not reliably preserve global structure, meaning distances between clusters in the visualisation may not reflect actual dissimilarities in the original high-dimensional space [76].

Figure 2.1 shows a comparison of SNE and t-SNE on the MNIST2500 dataset, where each point represents a handwritten digit from 0 to 9. The colour indicates the true class label. As illustrated in Figure 2.1, t-SNE mitigates the crowding problem by using a heavy-tailed t-distribution in the low-dimensional space, in contrast to SNE, which uses a Gaussian distribution.

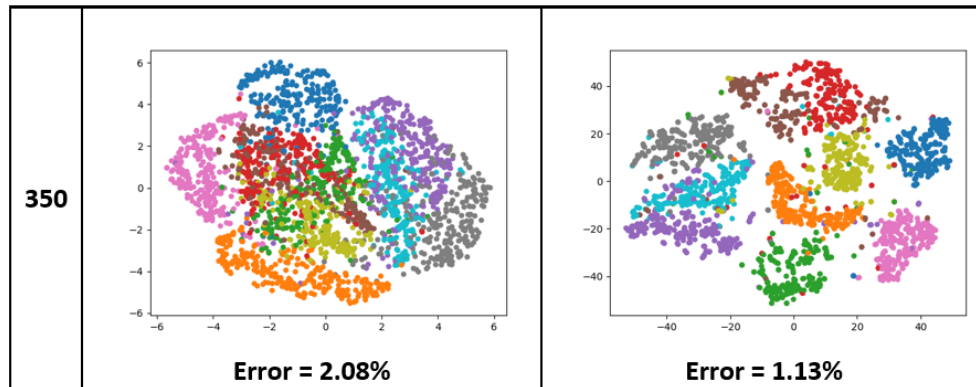


Figure 2.1: Comparison of SNE (left) and t-SNE (right) on the MNIST2500 dataset [69].

2.5 Numerical Instability with Floating Point Numbers

Numerical instability often occurs during calculations with floating point numbers due to the limited precision in representing real numbers in computers. Such instabilities can lead to rounding errors, loss of significant digits, and inaccurate results, especially in operations like subtracting nearly equal numbers or multiplying very large or very small numbers. The machine epsilon ϵ is often used to describe the upper bound of the rounding error. This value is also the smallest number that can be added to 1 without losing data. The numerical value depends on the standard used, but for 64-bit floating point numbers, the value is $\epsilon = 2^{-53}$ [41].

2.6 Frameworks

This section presents the environments and frameworks used throughout the study. It begins with an overview of the Gymnasium library, which provides standardised interfaces for reinforcement learning environments. The environments investigated, Cart-Pole and Lunar Lander, are then described in detail, including their respective observation and action spaces. Finally, the reward functions that guide learning in each environment are outlined.

2.6.1 Gymnasium

Gymnasium is an open-source library that provides a standardised API for RL environments. It makes it easier for different environments and training algorithms to work together. By offering user-friendly environments and customisation tools, Gymnasium improves the reproducibility and reliability of RL research. This allows researchers to focus on innovation rather than implementation details. The standardised approach also simplifies the development and testing of RL algorithms, helping to accelerate progress in the field [70].

2.6.2 Cart-Pole

The Cart-Pole problem, popularised by Barto, Sutton, and Anderson [3], involves balancing a pole on a moving cart by applying forces to the cart, making it a standard testbed for evaluating RL algorithms. Its simplicity makes it easy to compare agent performance and demonstrate key RL concepts. Additionally, its implementation in OpenAI's Gymnasium library provides a well-documented and easily accessible platform, making it convenient for rapid experimentation and reproducibility [45], [55], [81] [16].

An illustration of the Cart-Pole environment is shown in Figure 2.2. The environment is fully observable, deterministic, and static, with a discrete action space and a continuous state space.

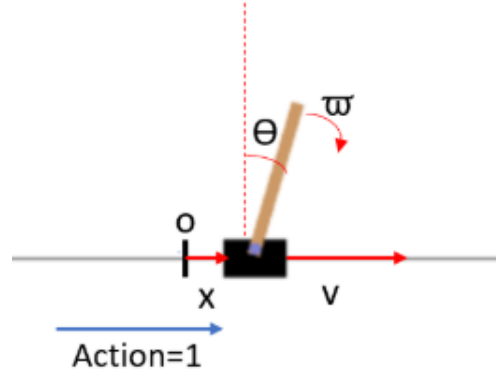


Figure 2.2: An illustration of the Cart-Pole environment [80].

2.6.3 Observation- and Action Space for Cart-Pole

In the Cart-Pole environment, the observation space consists of four variables. The observation space, which includes variables such as cart position and pole angle, is summarised in Table 2.1.

Num	Observation (Symbol)	Range
0	Cart Position (x)	$[-4.8, 4.8]$
1	Cart Velocity (v)	$[-\infty, \infty]$
2	Pole Angle (θ)	$[\approx -0.418 \text{ rad } (-24^\circ), \approx 0.418 \text{ rad } (24^\circ)]$
3	Pole Angular Velocity ($\bar{\omega}$)	$[-\infty, \infty]$

Table 2.1: Observation space for the Cart-Pole environment [16].

The action space consists of two discrete actions [16]:

- Action 0: Push the cart to the left.
- Action 1: Push the cart to the right.

2.6.4 Lunar Lander

The Lunar Lander environment introduces additional challenges, such as a larger state space, complex dynamics, and more precise control requirements. In Lunar Lander, the agent must learn to land a spacecraft between two flags without crashing, making it a popular benchmark for evaluating RL algorithms in more realistic scenarios [19], [59], [2], [47], [57], [17]. An illustration of the Lunar Lander environment is provided in Figure 2.3.

Similar to Cart-Pole, the Lunar Lander environment is fully observable, static, and features a discrete action space with a continuous state space. However, unlike Cart-Pole, it is stochastic by default due to randomness in the simulation dynamics.

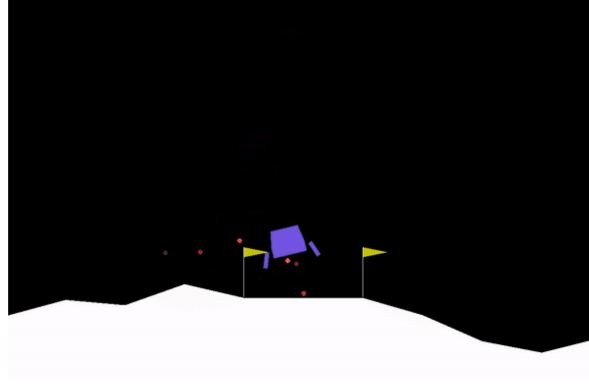


Figure 2.3: A visualisation of the Lunar Lander environment [8].

2.6.5 Observation- and Action Space for Lunar Lander

In the Lunar Lander environment, the observation space consists of eight variables. The eight-dimensional observation space of the Lunar Lander environment is detailed in Table 2.2.

Num	Observation (Symbol)	Range
0	X Position (x)	$[-2.5, 2.5]$
1	Y Position (y)	$[-2.5, 2.5]$
2	X Velocity (v_x)	$[-10, 10]$
3	Y Velocity (v_y)	$[-10, 10]$
4	Lander Angle (θ)	$[-2\pi, 2\pi]$
5	Angular Velocity ($\bar{\omega}$)	$[-10, 10]$
6	Left Leg Contact ($l_contact$)	$[0, 1]$
7	Right Leg Contact ($r_contact$)	$[0, 1]$

Table 2.2: Observation space for the Lunar Lander environment [17].

The action space consists of four discrete actions [17]:

- Action 0: Do nothing.
- Action 1: Fire left orientation engine.
- Action 2: Fire main engine.
- Action 3: Fire right orientation engine.

2.6.6 Reward Functions

Cart-Pole

The goal in Cart-Pole is to keep the pole balanced upright by moving the cart left or right. The environment gives a reward of +1 for every time step the pole remains upright. The episode ends when the pole falls past a certain angle, the cart moves out of bounds, or the agent reaches a maximum number of steps, 500 for CartPole-v1, which was used in this study. There is no terminal reward - the total reward is simply the sum of +1s collected during the episode [16].

Lunar Lander

In Lunar Lander, the agent receives a more complex and shaped reward signal. For each step, the reward is:

- increased or decreased based on the lander's distance to the landing pad.
- increased or decreased based on the lander's vertical and horizontal speed.
- decreased when the lander is tilted (i.e., the angle is far from horizontal).
- increased by 10 points for each leg that touches the ground.
- decreased by 0.03 points for every frame a side engine fires.
- decreased by 0.3 points for every frame the main engine fires.

At the end of an episode, an additional reward is applied:

- -100 points for crashing.
- +100 points for landing safely.

An episode is considered successful if the total score reaches at least 200 points. This reward shaping encourages not only successful landings but also efficient and precise control [17].

2.7 Biological plausibility

The human brain, despite having an overall high energy demand, is relatively speaking very cost-efficient [56]. Although the human brain is still not fully understood, there are many processes in the brain that have been thoroughly studied. Prominent breakthroughs in machine learning have, in large part, built on these processes and replicated them in some sense through code. This concept of creating artificial intelligence that can be related to biological intelligence is referred to as biological plausibility

[5].

2.8 Sustainability

One of the principal advantages of model-based reinforcement learning agents is their improved sample efficiency, which enables them to achieve target performance levels using fewer environment interactions [11]. This characteristic has important implications for sustainability, as it reduces the computational resources and energy consumption required for training. For instance, Gao and Wang [20] demonstrated that model-based adaptations of Double DQN and Soft Actor-Critic (SAC) converged in significantly fewer episodes compared to their model-free counterparts, thereby saving thousands of environment steps during training.

2.8.1 Goals and Frameworks

The United Nations Sustainable Development Goals (SDGs) encourage sustainable innovation and reduced resource consumption. SDG 12 (responsible consumption and production) calls for more efficient use of resources and sustainable practices [13], and SDG 13 urges efforts to reduce greenhouse gas emissions in all sectors, including tech [14].

Research that reduces energy consumption and computational efficiency aligns with Norwegian national climate ambitions [67]. Norway has set targets for emission reductions (aiming for carbon neutrality by 2030 and, at the latest, 2050), so research that lowers energy demand and improves computational efficiency, however modestly, contributes toward the tech community's share of climate solutions.

3. Related Work

This chapter reviews prior research relevant to the present study, focusing on two key areas. Section 3.1 examines how reinforcement learning (RL) algorithms have been applied in low-dimensional environments such as Cart-Pole, with particular attention to widely used methods like Q-learning and Deep Q-Networks (DQN). Section 3.2 focuses on sample efficiency, specifically in high-dimensional domains such as Atari games, humanoid control tasks, and board games like Chess and Go. These domains were chosen because sample efficiency is particularly critical in such settings and represents areas where most of the current RL research is conducted and where the most relevant studies are available.

3.1 Reinforcement Learning in Low-Dimensional Environments

This section focuses on the use of model-free RL algorithms in low-dimensional control environments. Model-based approaches are typically not benchmarked in such settings, as their primary advantage, sample efficiency, offers limited benefit when interacting with simple and inexpensive environments. This review emphasises foundational model-free algorithms such as Q-learning and Deep Q-Networks (DQN).

Q-learning, introduced in Section 2.1.7, is one of the most fundamental model-free RL algorithms. While it has been applied to the Cart-Pole environment, few peer-reviewed studies report successful implementations. Kumar [45] claims to solve the CartPole-v0 environment in approximately 300 episodes. Unlike CartPole-v1, used in the present study, CartPole-v0 limits episodes to 200 time steps, and Kumar defines success as an average reward of at least 195 over 100 episodes. However, attempts to reproduce these results using similar code and hyperparameters (see Appendix A) yielded noticeably lower performance. In addition, the referenced code repository fails to execute and lacks documentation, raising concerns about the reproducibility and reliability of the findings.

DQN, an extension of Q-learning, is one of the most widely used model-free RL algorithms; it is described in more detail in Section 2.1.7. Compared to Q-learning, DQN has been more frequently applied to low-dimensional RL problems. Kumar [45] also employed DQN to solve CartPole-v0 within 200 episodes. Other studies [54], [77] suggest that DQN generally requires approximately 0.5×10^5 environment steps, which translates to ≈ 250

episodes at most for CartPole-v0, to consistently achieve a reward greater than 200 per episode, which makes Kumar’s DQN result more plausible. Wong et al. [77] reported solving CartPole-v1 after approximately 2×10^5 steps, while Müller and Kudenko [54] observed that their DQN-agent did not exceed a reward of ≈ 200 after the same number of steps without reward shaping. This discrepancy suggests that variations in the implementation, such as reward shaping and hyperparameter configurations, may contribute heavily to the differences in performance across studies using the same algorithm and environment.

Policy gradient methods, introduced in Section 2.1.7, typically require a large number of environment interactions to converge to an optimal policy. Proximal Policy Optimization (PPO) is among the most widely adopted methods in this category and has demonstrated strong performance in environments with discrete action spaces such as Cart-Pole [16]. Kölle et al. [42] report that PPO consistently achieves the maximum episode reward of 500 after approximately 5×10^5 environment steps. In contrast, Jin et al. [38] achieve this performance with only 0.4×10^5 steps, while Wong et al. [77] report convergence at around 2.0×10^5 steps. All three studies employ the same actor-critic architecture originally proposed by Schulman et al. [62]; however, they differ in hyperparameter settings, including batch size, learning rates, the discount factor γ , and the GAE parameter λ . Jin et al., in particular, use a lower value for both γ and λ , as well as higher learning rates for the actor and critic, which may explain the improved sample efficiency observed in their implementation.

3.2 Sample Efficiency in Reinforcement Learning

Although RL has made impressive progress across a range of domains, from classic control environments to complex games like Atari and Go, a persistent challenge has been its poor sample efficiency, particularly in real-world or high-dimensional settings. This limitation has motivated a wave of research aimed at reducing the amount of interactions with the environment required to learn effective policies. Recent work has explored a variety of approaches to this problem, including model-based methods that learn a predictive model of the environment, model-free methods that leverage data augmentation and representation learning, and hybrid techniques that combine planning with policy learning. These advances have significantly narrowed the gap between academic RL and practical applications, especially in low-data regimes.

Kaiser et al. [40] introduced Simulated Policy Learning (SimPLE), a model-based deep RL algorithm that simulates future frames from raw pixel in-

puts. Policies are trained using imagined rollouts within this learned model, allowing the agent to learn Atari games with as few as 100K interactions and roughly two hours of gameplay. Notably, SimPLe manages to outperform popular model-free RL algorithms such as Rainbow and PPO even when those baselines are given twice as many interactions (200K), highlighting the superior sample efficiency of model-based approaches in low-data regimes. While SimPLe demonstrated impressive sample efficiency, it also revealed common challenges in MBLR: unstable training, high variance across runs, computational overhead, and lower final performance compared to strong model-free baselines like Rainbow DQN [31].

Since inaccuracies and brittleness in the learned model can significantly degrade policy learning and reduce sample efficiency, several works have focused on improving model robustness by explicitly accounting for uncertainty. Chua et al. [12] introduced PETS, a probabilistic ensemble of neural networks, where multiple models are trained to capture epistemic uncertainty, allowing for more reliable planning in continuous control tasks. Their results showed that PETS could match the asymptotic performance of state-of-the-art model-free methods like SAC and PPO while requiring dramatically fewer samples - up to 8 times fewer than SAC and 125 times fewer than PPO on the Half-Cheetah task. Similarly, Kurutach et al. [46] and their ME-TRPO demonstrated that ensemble-based models, which combine predictions from multiple learned dynamics models to mitigate model bias, substantially improve stability and generalisation in low-data regimes. Their approach achieved comparable performance to strong model-free methods such as TRPO and PPO while requiring approximately 100 times fewer samples, making it one of the first purely model-based methods capable of solving complex high-dimensional tasks like Humanoid.

Building on these insights, the MuZero algorithm, proposed by Schrittwieser et al. [61], takes a different approach by avoiding explicit state prediction altogether. Instead, it learns a model in latent space that predicts only the quantities needed for planning: reward, value, and policy - using Monte Carlo tree search (MCTS). This approach bypassed the complexity of pixel-level prediction and achieved state-of-the-art performance on Atari while also generalising to domains like chess and Go, all without access to the true environment dynamics. MuZero's success showed that accurate dynamics modelling is not always necessary if the learned model is directly optimised for decision-making. In Atari, MuZero outperformed the previous state-of-the-art model-free method R2D2 in 42 out of 57 games and surpassed the best prior model-based approach, SimPLe, across the board. Additionally, a more sample-efficient variant, MuZero Reanalyze, achieved a 731% median normalised score using only 200 million frames, significantly outperforming strong model-free baselines like Rain-

bow (231%), IMPALA (192%), and LASER (431%). In the game of Go, MuZero even slightly outperformed AlphaZero despite using fewer residual blocks (16 vs. 20), suggesting that it effectively reuses computation within its planning process to gain a deeper understanding of game states. These results demonstrate the impressive performance and the sample efficiency that model-based approaches can achieve when carefully aligned with decision-making objectives.

Using MuZero’s latent dynamics and MCTS framework, Ye et al. [83] introduced EfficientZero in 2021, pushing the limits of sample-efficient reinforcement learning by integrating improved training stability, data-efficient network designs, and a unified planning loop that combines policy and value learning. Unlike MuZero, which was designed for high-performance settings with extensive computation and data, EfficientZero is optimised for learning from minimal environment interaction. It achieves 194.3% mean and 109.0% median human performance on the Atari 100k benchmark, making it the first algorithm to reach superhuman performance in Atari games with only two hours of gameplay. In addition, it outperforms model-free baselines such as SAC on the DMControl 100k benchmark and approaches the performance of DQN at 200 million frames using 500× less data. Building on this foundation, a follow-up by different authors introduced EfficientZero V2 in 2024, which further improves sample efficiency and generalisation across both vision-based and low-dimensional control tasks [73]. As of 2025, EfficientZero V2 represents the most sample-efficient reinforcement learning algorithm, setting a new benchmark for performance in constrained-data environments. Both EfficientZero and EfficientZero V2 are open-source, supporting reproducibility and enabling broader adoption of MCTS-based reinforcement learning techniques in the research community.

Hafner et al. [29] introduced PlaNet, a model-based agent that learns a compact model of the environment from images and plans its actions in this learned internal space using the Cross-Entropy Method (CEM). At the core of PlaNet is a Recurrent State Space Model (RSSM), which helps the agent remember important past information through a recurrent network and handle uncertainty about what might happen next using random variables. By simulating thousands of possible future trajectories within this internal model, PlaNet can evaluate action sequences without interacting with the real environment, enabling more accurate planning and dramatically improved sample efficiency even with limited experience. Despite being trained with only 1,000 environment interactions per task, PlaNet achieved competitive or superior performance to model-free agents like A3C and D4PG, which required up to 100× more data. For example, on the DeepMind Control Suite, PlaNet demonstrated data efficiency gains of up to 500× compared to D4PG while achieving similar or better returns in

tasks such as Cheetah, Ball-in-Cup, and Walker.

Expanding on PlaNet, Hafner et al. [28] developed Dreamer, which replaces planning with policy learning and further improves sample efficiency by training policies entirely from imagined rollouts in the learned latent space. This process, referred to as dreaming, enables the agent to learn not from real interactions but from model-generated sequences. Dreamer maintained the data efficiency of PlaNet while substantially improving final performance. After only 5 million environment steps, it reached an average performance of 823 across tasks, outperforming PlaNet (332) and even surpassing the best model-free agent D4PG (786), which required over 100 million steps. These results highlight the potential of latent imagination to scale sample-efficient learning to more complex continuous control problems.

Janner et al. [37] proposed Model-Based Policy Optimization (MBPO), a hybrid approach that combines the sample efficiency of model-based learning with the stability of model-free methods. Rather than performing long model rollouts that accumulate error, MBPO generates short synthetic trajectories using a learned dynamics model and uses them to train a model-free policy via SAC. This approach significantly reduces the impact of model inaccuracies while retaining the benefits of sample efficiency from imagination-based training. On standard continuous control benchmarks, MBPO achieved asymptotic performance comparable to state-of-the-art model-free methods like SAC but with substantially fewer environment interactions. For instance, on the Ant task, MBPO matched SAC's final performance using only 300,000 steps - an order of magnitude fewer than the 3 million steps required by SAC.

Sekar et al. [64] introduced Plan2Explore, a self-supervised model-based reinforcement learning approach that enables agents to explore their environment and learn a global world model without access to task-specific rewards. The agent builds this model entirely through intrinsic motivation - driven by maximising expected information gain - and later uses it to solve new tasks in a zero-shot or few-shot manner. Unlike traditional RL methods that rely on external rewards during training, Plan2Explore learns behaviours purely from its own curiosity, making it highly scalable and data-efficient. On the DeepMind Control Suite, it achieved state-of-the-art zero-shot performance from raw pixels across a range of tasks, and its few-shot adaptation rivalled or even surpassed the performance of Dreamer, a leading supervised agent. Notably, Plan2Explore's ability to generalise to multiple downstream tasks, where agents trained with task-specific rewards, such as Dreamer, struggled, highlights the value of learning task-agnostic world models in low-reward or reward-sparse environments.

In addition to the methods discussed above, several recent algorithms have also demonstrated strong performance under constrained data regimes. TD-MPC [30] combines model predictive control (MPC) with temporal difference (TD) learning in a hybrid model-based/model-free framework. It uses a learned dynamics model for short-term trajectory optimisation and a value function to estimate long-term returns, achieving strong performance on visual control benchmarks with significantly improved sample efficiency over prior methods. CMBAC, proposed by Wang et al. [74], uses an ensemble of models comparable to PETS [12] and ME-TRPO [46] and achieves significantly better sample efficiency and improved robustness in noisy environments compared to earlier state-of-the-art approaches. Similar to PlaNet [29] and Dreamer [28], IRIS [49] learns a latent world model in which the agent can train itself with minimal environment interaction. With minimal tuning, IRIS outperforms several recent methods on the Atari 100k benchmark and achieves a mean human-normalized score of 1.046, reaching superhuman performance on 10 out of 26 games after just two hours of experience. Sample efficiency in reinforcement learning is typically higher when agents learn from low-dimensional state information rather than raw pixel inputs. To bridge this gap, methods such as CURL [65] and DrQ [43] improve the sample efficiency of model-free RL from visual observations by learning more robust representations. CURL achieves up to $1.9\times$ better performance on the DeepMind Control Suite, while DrQ significantly boosts data efficiency on the Atari 100k benchmark. An improved version, DrQ-v2 [82], further refines the approach and achieves state-of-the-art performance among model-free methods on several pixel-based benchmarks, including human-level performance on Atari.

As of 2025, EfficientZero V2 [73] remains the most sample-efficient reinforcement learning algorithm, particularly on benchmarks like Atari 100k. However, in 2023, Max Schwarzer et al. introduced Bigger, Better, Faster (BBF) [63], a model-free approach that significantly narrows the gap between model-free and model-based methods in terms of sample efficiency. BBF builds on prior model-free techniques by combining larger models, better data augmentation strategies, and improved bootstrapping mechanisms. Despite being model-free, BBF achieves performance that closely rivals EfficientZero V2 on Atari 100k, making it the most competitive model-free alternative to date in low-data regimes.

4. Method

The purpose of this study was to investigate biological plausibility in reinforcement learning, mainly through the application of vector quantisation methods to a model-based reinforcement learning agent (MBRL). This chapter will present the methodology of the study and give a short presentation of the experiments performed. The study followed the basic hypothetico-deductive model of forming a hypothesis, predicting the consequences of it, and then attempting to falsify it through experiments. The data was analysed quantitatively through graphs, which were compared with each other.

4.1 Choice of Technology and Development methods

This section presents the choices of technologies made throughout the study, with a brief description of why they were chosen. It includes an overview of the environments, libraries, and frameworks selected for implementation, as well as the research approach.

4.1.1 Programming Language

Python was chosen as the main programming language for this project because of its strong support for machine learning and reinforcement learning. It has a large ecosystem of libraries that make it easier to build and test agents, some of which are mentioned in Section 4.1.2. In addition, Python provides access to several widely used environments and tools for reinforcement learning, such as OpenAI’s Gymnasium, which was used in this project (see Section 2.6). Python’s flexibility, community support, and availability of libraries made it the most suitable choice for developing and evaluating a model-based reinforcement learning agent.

4.1.2 Libraries

The implementation and analysis in this study relied on several widely used Python libraries. `NumPy` was employed for efficient numerical operations, particularly vector and matrix computations, which contributed to code clarity and performance. `SciPy` was utilised primarily for its implementations of the softmax and log-softmax functions, as well as for spline-based interpolation in relevant experiments. `Scikit-learn` served as the

primary library for regression modelling and clustering algorithms, providing standardised implementations used across multiple experiments. Finally, `Matplotlib` was used to visualise the results, including the generation of figures and plots presented in the results Section 5.

4.1.3 Environments

The experiments were performed primarily in the Cart-Pole environment presented in Section 2.6. This environment had several properties that were beneficial for this study. First, the relatively few dimensions in the state space allowed for quicker experiments. Additionally, the deterministic nature of the environment meant that the exploration rate could remain mostly fixed throughout the study, eliminating a variable from the experiments. Lastly, the discrete action space allowed for a simpler policy structure, enabling quicker prototyping.

The Lunar Lander environment was also used to a lesser extent, as this environment required more episodes for the agent's performance to stabilise. Due to the nature of the study, experiments in Lunar Lander were not prioritised.

4.1.4 Research Method and Collaborative Framework

The study followed an experimental research process aimed at investigating the impact of biologically plausible methods. The research process was structured according to the hypothetico-deductive model. Each research cycle involved forming a hypothesis, implementing the necessary changes, running experiments, and analysing the results using visualisation tools. This flexible, iterative approach was essential to address the open-ended nature of the research question and allowed the model and experimental setup to evolve in response to new findings.

Division of Work and Roles

The work was carried out collaboratively, with responsibilities distributed to ensure both technical progress and effective project management. Roles were assigned based on the natural division of tasks and to support structured workflow throughout the semester.

Technical Responsibilities

Literature review and theoretical background: One member was responsible for researching relevant literature and summarising theoretical concepts such as biological plausibility, model-based reinforcement learning and vector quantisation.

Implementation and experimentation: Another member focused on developing the agent, integrating vector quantisation techniques, and conducting experiments in the selected environments.

Data analysis and visualisation: One member took the lead in analysing simulation results and producing figures to support interpretation and comparison.

Report writing: All members contributed to writing, with sections distributed according to expertise and technical involvement.

Project Coordination Roles

Meeting coordinator: One member was responsible for scheduling group meetings, sending meeting notices via email and ensuring attendance.

Meeting secretary: Another member was responsible for writing and distributing the meeting minutes.

Document manager: Oversaw the version control of the written report and ensured that the mandatory work requirements were delivered on time.

GitHub manager: Maintained the shared codebase, ensured commits followed conventions, created branch and pull request rules, and handled merging and issue tracking.

Communication and Documentation

Regular communication was maintained through digital platforms, primarily Discord, which enabled efficient day-to-day collaboration. Version control was handled using Git, with GitHub used as the central repository for code and documentation. The group met frequently to work together on the project, and weekly meetings with the supervisor were held to receive guidance and resolve open questions. Microsoft Teams was used for digital meetings with the study's supervisor when needed. Overleaf was used as a collaborative tool for writing the thesis, as it allowed for multiple contributors to work simultaneously.

4.2 Agent Design

This section will cover the design of the agent developed for this study. It will describe how the agent models its environment, how the agent's policy works, and the clustering process used for several experiments. To limit the scope of the study, the agent was developed for a discrete action space.

4.2.1 Model

Section 2.1.5 states that an MBRL agent creates an explicit model of the environment to form the basis of its decision-making process. The model in this study consisted of two main parts: a mapping of previously visited states to their respective rewards and a transition model. These states were, for the most part, unaltered from the Gymnasium framework covered in Section 2.6. During the policy calculations described in Section 4.2.2, however, the states were normalised. At the end of each episode, the agent stored all the states it had visited that episode- and assigned the sum of earned rewards to each. The rewards could be discounted, but this was not used in any of the experiments as it was not considered relevant to the research question. Transitions were modelled in two different ways, which will be discussed in Section 4.2.3.

4.2.2 Policy

The agent’s policy was an epsilon-greedy policy consisting of two steps. The first step was a form of the Bellman equation where the utility of the current state was decided based on the rewards attained in similar states. To define similarity, a Gaussian function on the form shown in equation 2.5 was used to calculate a weight for a given state $\vec{s}$, using the previously encountered states $\vec{\mu}$ as centres. In equation 4.1, the i index refers to the previously encountered states. For each previously encountered state μ_i , the respective Gaussian function f_i outputs a weight for it. The query points and centre used by the agent depended on the transition model.

$$f_i(\vec{s}) = \exp \left(-\frac{\left\| \frac{\vec{s} - \vec{\mu}_i}{\vec{s}_{std}} \right\|_2^2}{\sigma} \right) \quad (4.1)$$

In equation 4.1, σ is called the Gaussian width. This hyperparameter adjusts the drop-off of values as the distance between the points increases. The $\vec{s}_{std}$ value is a vector with the standard deviation for each dimension in the state space, calculated from the states currently in the agent’s model. Using this Gaussian function, the agent calculated a weighted average of previously attained rewards to designate a utility for the current state s and possible action a :

$$U_a(\vec{s}) = \frac{\sum_i r_{\mu_i} \cdot f_i(\vec{s})}{\sum_i f_i(\vec{s})} \quad (4.2)$$

Following this, the agent generated a random value $x \sim \mathcal{U}(0,1)$, which was compared to an exploration rate ϵ . Together with equation 4.2, the policy

of the agent during training can be described as follows:

$$a(\vec{s}) = \begin{cases} \text{random action where } U_a(\vec{s}) = 0 & \text{if } \exists a \text{ such that } U_a(\vec{s}) = 0 \\ \text{random action} & \text{if } x < \epsilon \\ \arg \max_{a \in A} U_a(s) & \text{otherwise} \end{cases} \quad (4.3)$$

After the training was complete, the agent would switch over to a greedy policy. The agent stopped exploring and simply chose the action that had the greatest predicted value.

4.2.3 Transition Model

As previously mentioned, two different transition models were implemented and tested. The agent could only use one model at a time, specified beforehand for a given experiment.

The first method was a simple mapping of state s_i to s_{i+1} for action a . After completing an episode, the agent would update its transition model with all the transitions it experienced. The agent would use the current state s and the previously encountered from-states s_i in the Gaussian functions. The weights from the Gaussian functions, as well as the rewards r associated with the to-states s_{i+1} , were used to perform the utility calculations. The reasoning was that the agent would compare its current state to the initial states previously encountered. If a previous from-state s_i were similar to the current state, there would be a good chance that selecting the same action would lead to a similar transition.

The second method involved using the previous changes in state Δs to form a prediction model and using this to estimate future rewards. A separate model was constructed for each action to predict the Δs for a given action and state using the methods presented in Section 2.2. This expected change was then added to the current state and used as the basis for the utility calculations. The predicted to-state for the current step was then compared to the previously experienced to-states s_{i+1} , representing the centres μ in the policy described in 4.2.2. In this case, the agent estimated where it would end up after an action using the collected data and would estimate a reward in this new state based on comparable states. For each time step in the training period, the predicted delta was compared to the actual delta, and the prediction model was updated if the mean squared error was too large.

4.2.4 Clustering

For this study, biological plausibility will be attempted by applying clustering algorithms to the agent's model of the environment inspired by

the biological process of sleep. The agent underwent clustering by first applying a vector quantisation technique to the model and reducing the set of states to a smaller representative amount. Additionally, each state was assigned a label to indicate which centroid it belonged to. A Gaussian function was then used to calculate a weighted average reward for the clusters, using the states assigned to the respective clusters as centres. The Gaussian width used here was the same as the one used in the policy.

The clustering methods tested were k-means, online clustering, and Gaussian Mixture models. Clustering was implemented with and without weighting, with the weighting function subject to experimentation to find an optimal method. All the clustering methods used were covered in Section 2.3.

Updating the transition model required different approaches depending on the transition model. From- and to-states were mapped to their respective clusters for direct transition mapping, with duplicate transitions discarded. In the delta transition model, each centroid was assigned a predicted delta for each possible action, calculated as a weighted average using a Gaussian function. To enhance computational efficiency, the to-states were pre-computed during this step.

4.2.5 Model-Free Comparisons

To provide a point of comparison, Q-learning and DQN learning agents were implemented and tested in a separate experiment. These agents shared some common hyperparameters with the model-based agent, so these were set to the same values as in the model-based agent. This enabled a direct comparison between model-based and model-free approaches to the same problem.

The Q-learning and DQN agents utilized an ϵ -greedy exploration strategy, with ϵ and other relevant hyperparameters held constant across both model-free and model-based variants to ensure comparability. During training, ϵ was fixed at 0.1 to maintain consistency with the model-based agents. In the evaluation phase, ϵ was set to 0, corresponding to a fully greedy policy. The discount factor γ was set to 1.0, implying that future rewards were not discounted, and the return corresponded directly to the total episodic reward. Both agents were trained and evaluated over an equal number of episodes, thereby ensuring that the total number of environment interactions, i.e., the sample budget, was equivalent across conditions.

All sources of environment dynamics, network initialisation, and action sampling were governed by random seeds. Each independent episode drew one seed. At the end of every trial, each agent wrote a JSON record containing its hyperparameter values and the sequence of per-episode

rewards for both training and evaluation. The hyperparameters for both agents can be found in the Appendix A.

4.3 Experimental Design

All of the experiments in the study were performed by setting up the desired parameters and then executing 100 runs of the experiment to produce a stable average of the results. One run consisted of a training period and a testing period, each consisting of 100 episodes. Each run was stored as a separate JSON file containing all the parameters, as well as the results of each individual episode. The runs were pseudo-randomized using a seed, which was also included in the test results. Note that the experiments also contained elements of randomness that were not controlled by the seed, so any reproduction of the experiments may have small variations from the ones presented here. The performance of the agent was evaluated based on average rewards achieved during the testing period.

4.3.1 Experiments

The following is a list of the experiments performed in this study. Section 4.2 covers the design of the different methods in detail, so this section will only provide a brief description of the setup of the experiment.

I. Direct Transition Model without Clustering

The agent used for this experiment used a basic transition model and did not employ clustering. This experiment formed a baseline for further experiments.

II. Direct Transition Model with Regular K-Means

Hypothesis: Compressing the agent’s model using regular k-means would produce equal or better performance.

Building on the previous experiment, regular k-means without weighting were implemented to investigate how this affected performance. The implementation of the clustering was performed as described in Section 4.2.4. Different values of k were tested to investigate the impact this had on performance.

III. Weighted K-Means

Hypothesis: Using weighting would improve performance over regular k-means.

Following the regular k-means, weighted k-means were implemented to investigate how weighting affected the results. The rewards for each state were initially raised to the third and fifth power and used as weights. Next, the softmax function was used to push the centroids closer to high-reward

states in an attempt to improve the agent's performance. However, due to the large number of states, it was suspected that this method may have been susceptible to numerical instability, as mentioned in Section 2.5. Therefore, variations using the logarithm of softmax were also tested.

IV. Direct Transition Model with Alternative Clustering Techniques

This experiment investigated the effects of alternative clustering techniques on performance. The experiment used the online clustering algorithm presented in 2.3.3, as well as a Gaussian Mixture Model as described in Section 2.3.2. For the online clustering, a filter was used during the reward calculations for the centroids instead of using labels, as was the case with k-means. The filter was a cut-off for the weights calculated by the Gaussian function, meaning that only the rewards of nearby states affected the calculations.

V. Delta Transition Model using Splines

Hypothesis: Modelling transitions as deltas would make for more accurate utility calculations and yield better results.

This experiment was the first to use the delta transition model, which was implemented using splines.

VI. Delta Transition Model using Linear Regression

Hypothesis: Using linear regression instead of splines to predict deltas would yield similar or better results while also reducing computational overhead.

A variation on the previous experiment where linear regression was used instead of splines to predict deltas.

VII. Weighted Clustering with Higher Gaussian Widths

Hypothesis: Larger Gaussian widths would improve post-clustering performance.

Clustering was investigated for larger Gaussian widths. This had not been previously done due to the relatively poor performance of the baseline. A larger value of σ was thought to improve performance, as larger values allow for less precise data. Both the delta and direct transition models were investigated.

VIII. Neuron Activation States

Hypothesis: Representing states in a more biologically plausible way would amplify the effects of clustering.

This experiment built further on the investigation of biologically plausible methods by representing states as a vector of neuron-like activations. These activations were calculated by dividing the initial dimensions into

discrete intervals and thereby multiplying the dimensions of the state vectors. Each initial dimension was mapped to a series of activations to each respective discrete value. The experiment used the delta transition model with linear regression. Another variation tested was to use positional encoding methods to represent states, as described in Section 2.1.7. Each dimension in the original state was transformed into a vector of four new dimensions. Positional encoding was also tested with a cut-off, where negative values were replaced by zero.

IX. DQN and Q-Learning Comparative Experiment

Hypothesis: Model-free agents would perform worse than model-based variants under low sample conditions. Q-learning was expected to underperform due to the high number of unique states and its reliance on sufficient exploration.

This experiment benchmarked a DQN and a Q-learning agent using only 100 training episodes. The goal was to evaluate their ability to generalise under a strict training budget and see how they perform alongside model-based variants. Q-learning was expected to require significantly more episodes to be comparable.

4.4 Sustainability Assessments

This study focused on applying vector quantisation techniques to MBRL and thus aimed to improve sample efficiency. These approaches were inspired by the energy-efficient mechanisms of the human brain and offer an alternative to computationally intensive deep learning systems. Although AI systems - particularly those based on deep learning - have come under scrutiny for their high energy consumption and limited sustainability, as demonstrated in NLP-focused studies such as Strubell et al. [68], this work contributes to addressing such concerns more broadly by improving the sample efficiency of reinforcement learning agents. In practice, better sample efficiency means the agent reaches a good performance with fewer training episodes or interactions, directly reducing the number of computations required, as presented in Section 2.8. This efficiency is particularly useful in robotics, such as the applications described in Section 2.1.6, where the energy required for interactions is even greater. While no formal sustainability assessment methods were applied, the project was conducted with attention to reducing computational load, increasing accessibility, and supporting the broader environmental goals mentioned in Section 2.8.1.

5. Results

This section will present the results gathered during the study. The results are organised as a separate subsection for each experiment performed, with variations of the experiment where relevant. The most important hyperparameters are discussed where relevant, but a full listing of the hyperparameters used for each experiment and variation is found in appendix A.

5.1 Direct Transition Model without Clustering

For the initial experiment, the agent was implemented using the direct transition model. Different variations were tested to investigate the impact of the Gaussian width on the performance of the model. These results formed a baseline for comparisons with later methods implementing clustering and weighted clustering.

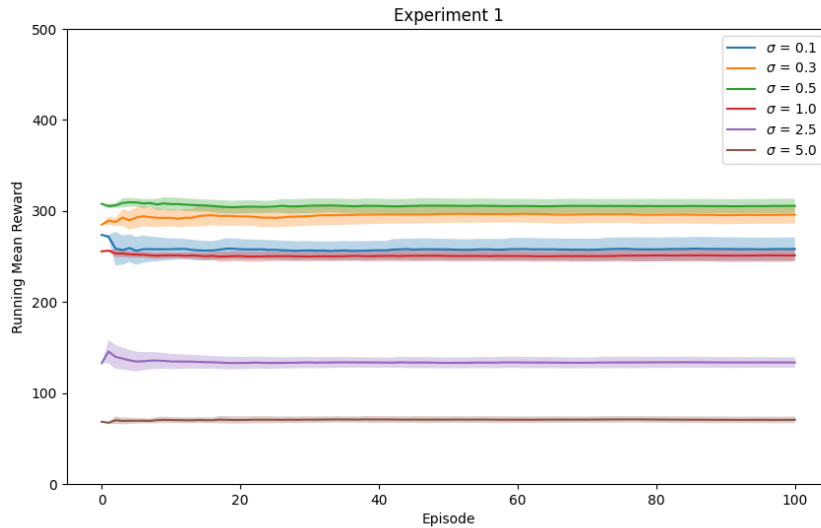


Figure 5.1: Running mean and standard deviation for the different variations of the experiment

Figure 5.1 shows the results of the six variations that were run. The graph shows that a σ in the range $[0.1, 1]$ produced significantly higher average

rewards than the values 2,5 and 5. Performance increased from $\sigma = 0.1$ up to a peak at $\sigma = 0.5$ and then declined as σ increased. A Gaussian width of $\sigma = 0.5$ produced the best mean rewards of the variations tested, with a running mean of around 300.

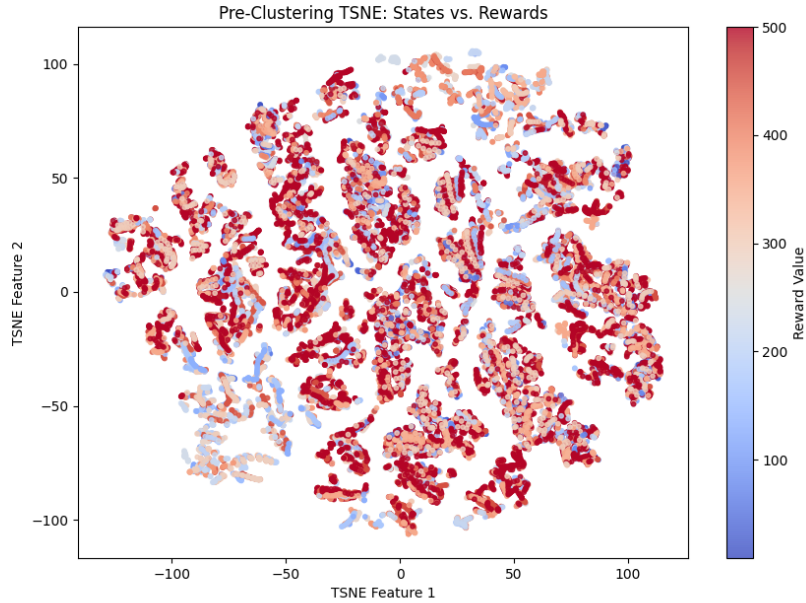


Figure 5.2: t-SNE plot of the agent's states after training

The above plot, Figure 5.2, shows an example run in a t-SNE plot, with states being mapped against their respective rewards.

5.2 Direct Transition Model with Regular K-Means

This experiment built on the previous experiment and was the first attempt at applying clustering to the agent. From the results of the previous experiment, it was deemed most reasonable to run the experiment using a Gaussian width σ of 0.5.

Finding the optimal value of k was done primarily by experimentation, using the elbow method to find an appropriate range. During experimentation, it was found that the agent would typically build up a model consisting of around 10,000 states after 100 episodes of training, varying depending on the performance of the agent during the run. For this reason, the k values in the range $[100, 5000]$ were investigated. Values greater than 5000

were tested but were not included, as performance was nearly identical for these values.

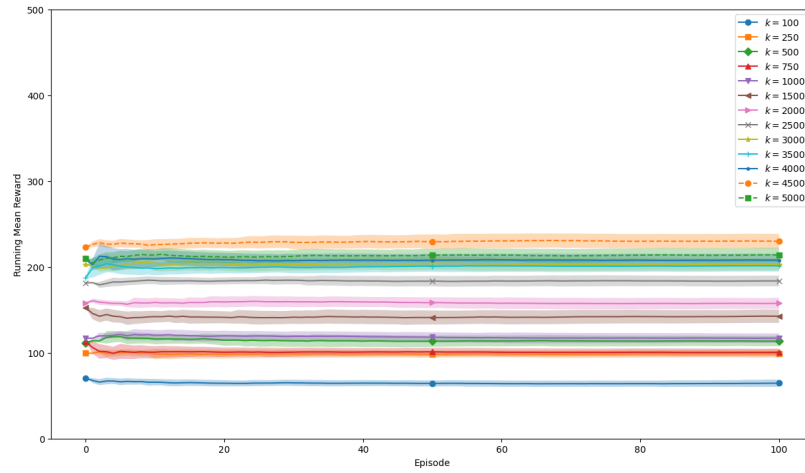


Figure 5.3: Results from the second experiment, with separate lines for each variation

Figure 5.3 shows the running mean and standard deviation for all the different values of k that were used. Values lower than 2500 were shown to have significantly poorer performance than the values in the range $[2500, 5000]$.

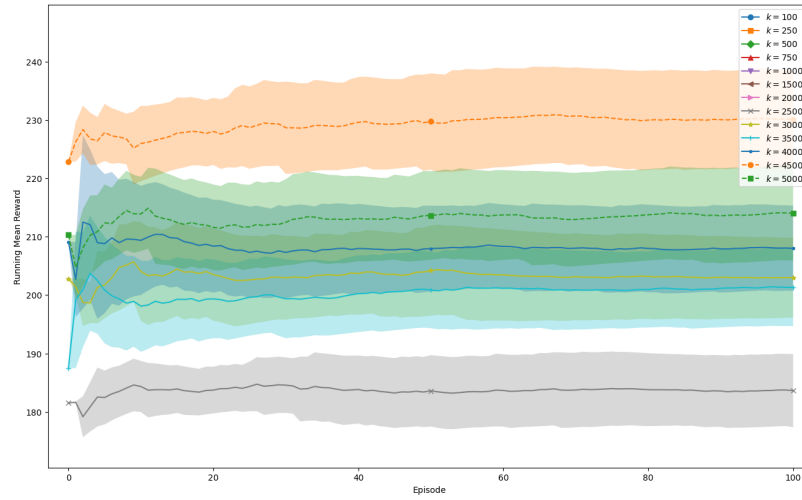


Figure 5.4: Zoomed in view of figure 5.3

Additionally, as made clearer in Figure 5.4, the agent performs worse with $k = 5000$ than $k = 4500$. As mentioned previously, k -values in the range $[2500, 5000]$ appear to produce higher averages than values outside this range. For subsequent experiments, the decision was made to use $k = 3000$, as this was the lowest value of the high-performing runs.

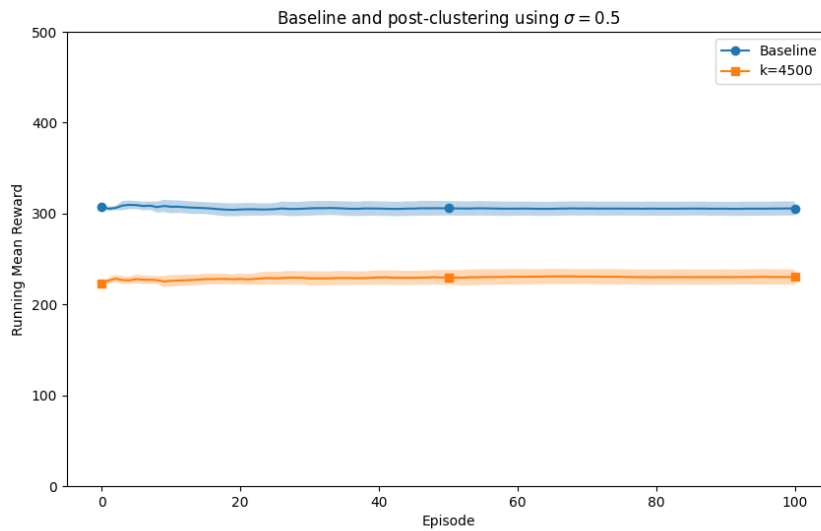


Figure 5.5: Comparison of experiments I and II

Figure 5.5 compares the baseline experiment to the results after clustering. Performance declines from around 300 in the baseline to around 220-250 after clustering.

5.3 Weighted K-Means

Following the experiments using regular k-means, a method of weighted k-means was tested using the rewards associated with the states as sample weights. The results from the previous experiments showed little improvement in performance for higher values of k , so $k = 3000$ was used throughout all variations of this experiment. As with the previous experiment, $\sigma = 0.5$ was used for the Gaussian functions.

The first variation used a simple power function, where the rewards were raised to a power and used as sample weight for the respective states. The results presented here were performed on the Cart Pole environment, meaning that all rewards were positive without scaling.

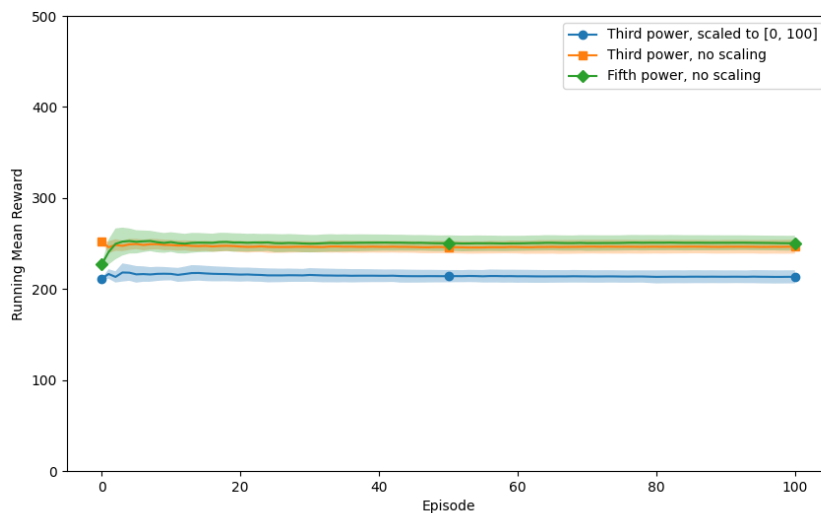


Figure 5.6: Direct transition model with exponential weighting

Figure 5.6 shows that raising the rewards to the third and fifth power produced nearly identical results, while scaling the rewards to the range $[0, 100]$ led to a lower mean reward after clustering. Both the variations without scaling produced a running mean of around 250.

The second variation used the softmax function on the raw rewards, where the outputs were used directly as sample weights. Variations using temperatures were also tested.

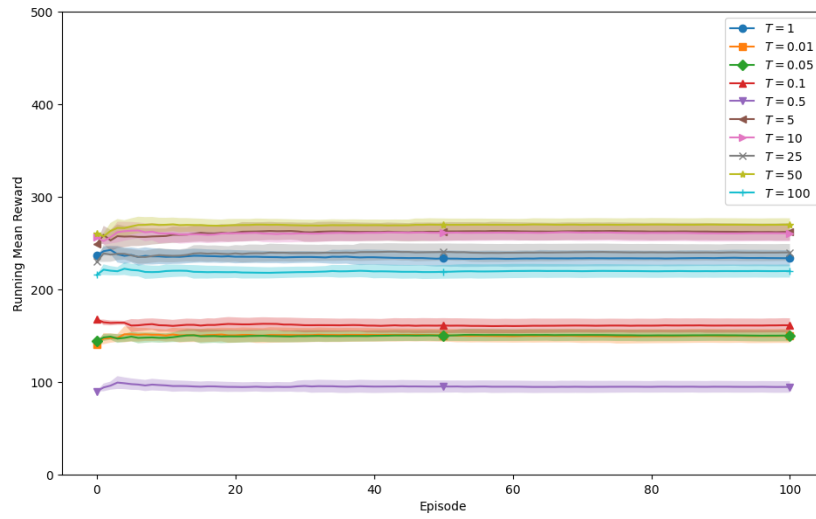


Figure 5.7: Direct transition model with softmax weighting

Applying the softmax function without temperature, shown as $T = 1$ in Figure 5.7, produced a running mean of around 240. For values $T < 1$, performance was worse than at $T = 1$, with the value $T = 0.5$ producing the lowest running mean of the variations at a running mean of around 100. Values $T > 1$ improved performance up to a peak at $T = 50$, after which performance decreased. The variation using $T = 100$ performed worse than at $T = 1$, so greater values were not attempted.

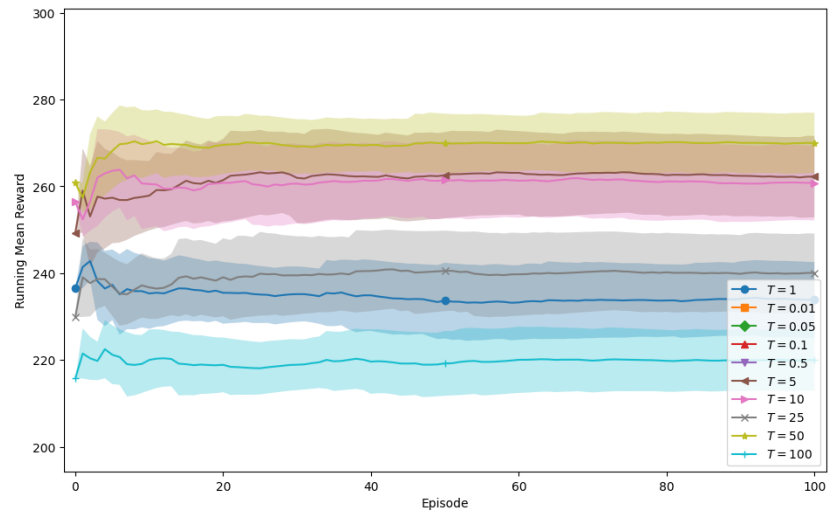


Figure 5.8: Closeup of figure 5.7

Finally, the log softmax function, the logarithm of softmax, was tested. The outputs of the log softmax function were shifted up to different upper bounds to investigate the impact this had on performance. After shifting the values, they were exponentiated.

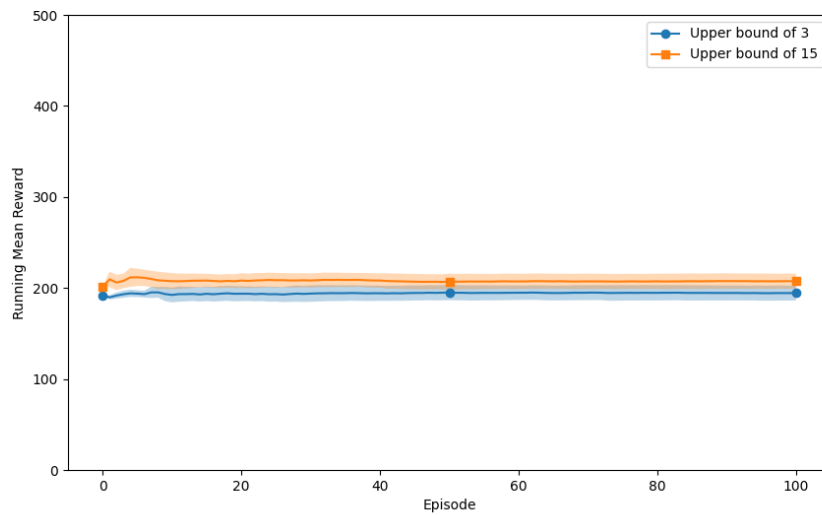


Figure 5.9: Log-softmax weighting

Both variations performed similarly, both producing running means of around 200. Further variations were briefly explored but largely produced similar results to Figure 5.9.

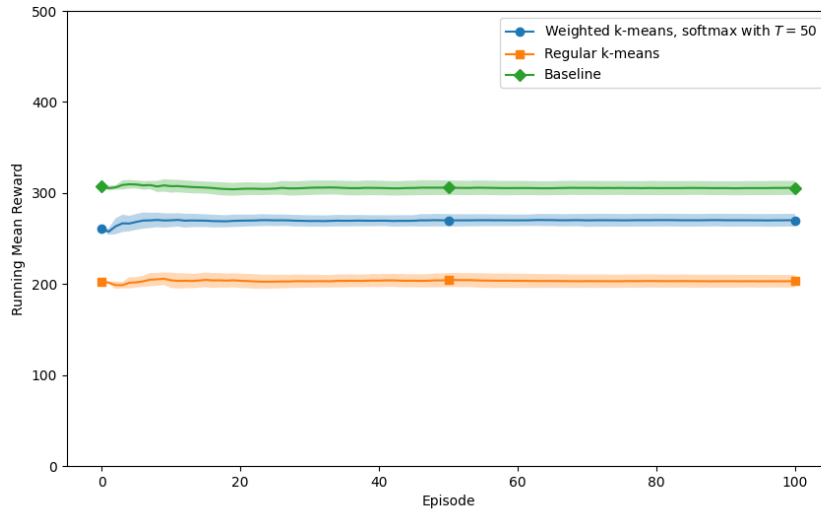


Figure 5.10: Comparison of experiments I, II and III

Figure 5.10 compares the results of the baseline, regular k-means and weighted k-means. Weighted k-means with softmax and temperature $T = 50$ is used for comparison as this variation produced the best performance of the variations tested.

5.4 Direct Transition Model with Alternative Clustering Techniques

Following the experiments using k-means clustering, the impact of alternative vector quantisation techniques on model performance was investigated.

5.4.1 Online Clustering

For this experiment, the online clustering algorithm presented in 2.3.3 was implemented and used in place of k-means. Both weighted and unweighted versions were tested. Variations of the experiment used different cut-off filters when calculating the rewards for the new centroids. Additionally, the effect of updating the centroids was investigated by running five epochs of 100 episodes, where the centroids were updated after each epoch. Finally,

a Gaussian width of $\sigma = 0.2$ was used for clustering and $\sigma = 0.3$ for the policy.

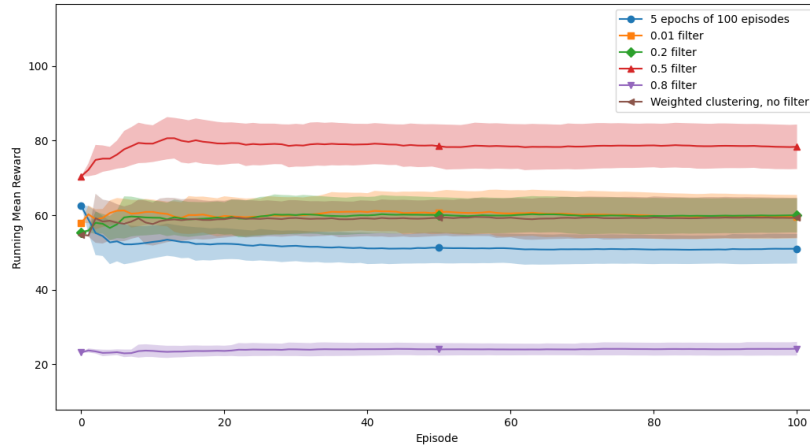


Figure 5.11: Variations of the experiment with online clustering. Note that the scale here is different due to the lower overall values.

Filters in the range $[0.01, 0.8]$ were tested and can be seen in Figure 5.11. The performance increases up to a peak with a running mean of around 80 when using 0.5 as a cut-off. Using 0.8 as a cut-off produced significantly worse results, with a running mean of around 20. The variation where centroids were updated was performed using a filter of 0.5, with weighted clustering using the softmax function with a temperature of 50. This variation performed worse than the equivalent variation that did not implement multiple epochs. Overall, the results of online clustering showed consistently poor performance.

5.4.2 Gaussian Mixture Model

The following experiment used the Gaussian mixture model (GMM) described in Section 2.3.2 to cluster the states after the initial training phase. Due to the high computational overhead when clustering with GMM, only the hyperparameters that had previously been deemed optimal were used: `n_components = 4500` and `sigma = 0.5`. Weighting was not attempted for GMM. The hyperparameters specific for GMM were: `covariance_type = "full"`, `random_state = 42` and `init_params = "kmeans"`.

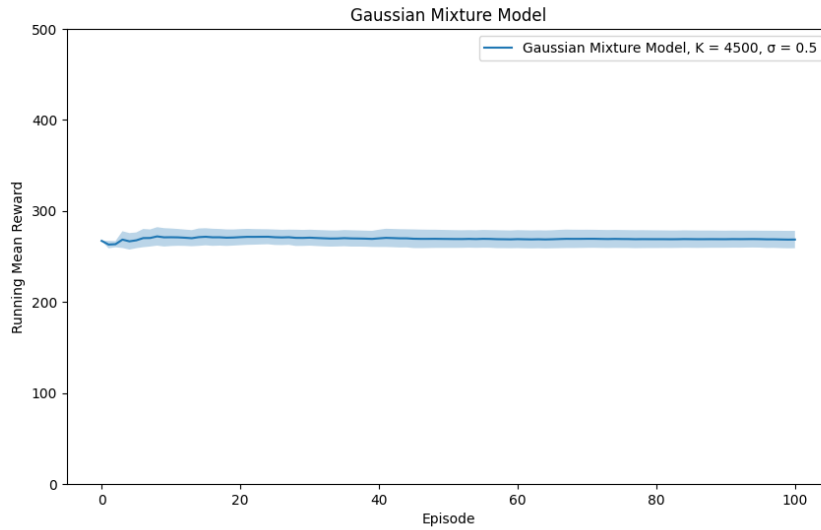


Figure 5.12: Running mean reward for Gaussian mixture model experiment

Figure 5.12 presents the performance of the agent when using the Gaussian Mixture Model (GMM) for clustering. The agent achieves a running mean reward of ≈ 270 , which significantly surpasses the performance of both online and standard k-means clustering. Moreover, it performs comparably to the weighted k-means configuration with a temperature parameter of $T = 50$, as illustrated in Figure 5.4.

5.5 Delta Transition Model using Splines

After replacing the direct transition model with the delta variant, a series of experiments were conducted with and without clustering to evaluate whether the new variant outperformed the original transition model.

The first experiment examined the effect of different Gaussian widths on model performance without clustering, establishing a baseline for comparison with clustered variants. In this experiment, SciPy's `RBFInterpolator` was used with the `thin_plate_spline` kernel to implement the prediction model.

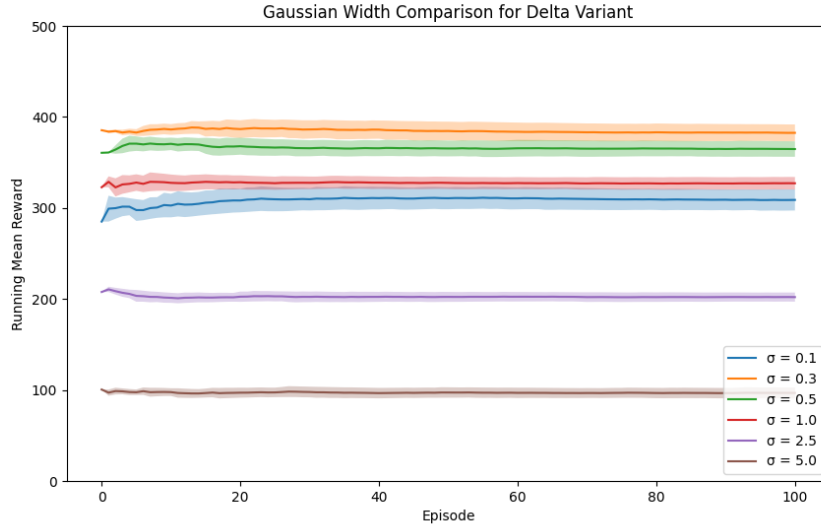


Figure 5.13: Comparison of different Gaussian widths using splines for the delta transition model, without applying clustering.

Figure 5.13 presents the results of six different Gaussian width configurations. The plot indicates that Gaussian widths in the range $\sigma \in [0.3, 0.5]$ yielded the highest average rewards. In contrast, values at or above $\sigma = 2.5$ resulted in substantially lower performance. Performance increased from $\sigma = 0.1$ to a peak at $\sigma = 0.3$, after which it declined as σ increased further. The best result was achieved with a Gaussian width of $\sigma = 0.3$, producing a running mean reward of approximately 385.

Following this experiment, the impact of varying the error threshold ϵ of the spline-based prediction model was evaluated. To isolate the effect of ϵ , all other hyperparameters were held constant at their previously optimised values from earlier experiments: the Gaussian width $\sigma = 0.5$, the number of clusters $K = 4500$, and the temperature parameter $T = 50$ used for softmax normalisation of rewards prior to clustering.

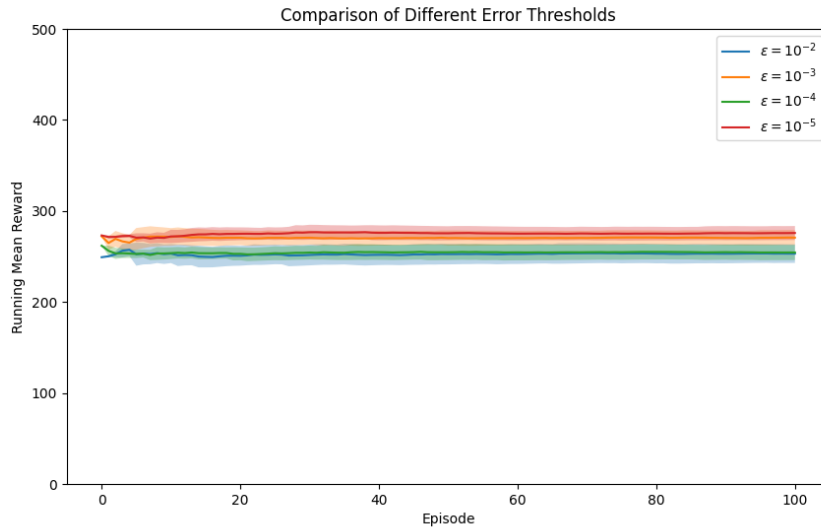


Figure 5.14: Comparison of different error thresholds for the Mean Squared Error (ϵ)

Figure 5.14 shows that $\epsilon = 10^{-2}$ and $\epsilon = 10^{-4}$ both result in running mean rewards of ≈ 254 , while $\epsilon = 10^{-3}$ yields ≈ 271 . The best performance is observed with $\epsilon = 10^{-5}$, achieving a running mean reward of ≈ 276 .

In the third experiment, the impact of varying the number of clusters K in the k-means algorithm was investigated. The hyperparameter configuration from the previous experiment was retained, with the error threshold being set to $\epsilon = 10^{-2}$.

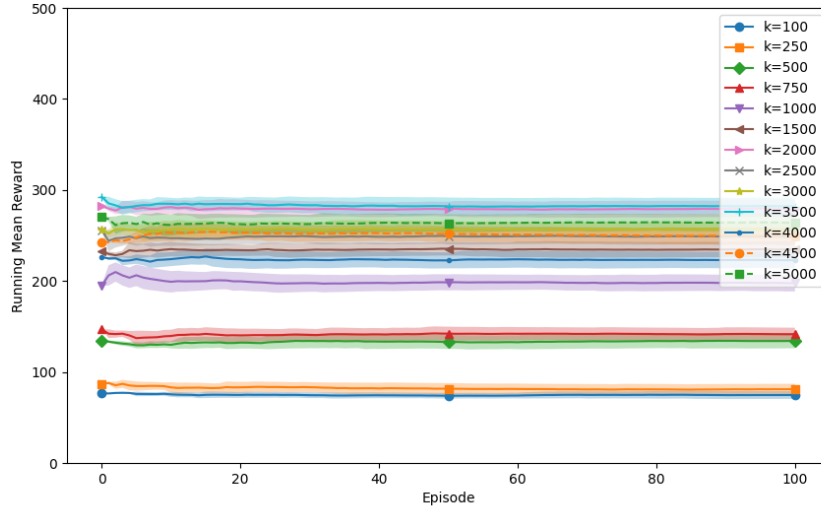


Figure 5.15: Comparison of different k values for the Delta Variant

As shown in Figure 5.15, the running mean reward increases steadily as the number of clusters K rises from 100 to 2000, reaching a local maximum at $K = 2000$. Performance declines slightly at $K = 2500$ and $K = 3000$, before improving again and reaching a comparable peak at $K = 3500$. Increasing K beyond this point does not yield further improvements, suggesting that performance saturates around these values.

5.6 Delta Transition Model using Linear Regression

This experiment was performed in the same manner as the previous experiment, only this time using linear regression instead of splines for predicting transitions. First, the different σ values for the Gaussian function were investigated without clustering to allow a clear comparison to the other experiments. Secondly, weighted clustering was performed using k-means, and different values of k were investigated. An error threshold of $\epsilon = 10^{-2}$ was used throughout this experiment due to the previous experiment showing this to be a sufficient value to balance performance and computational efficiency.

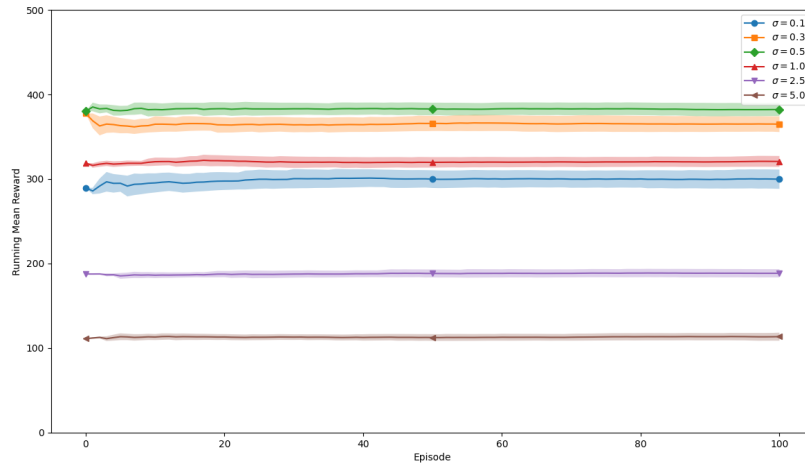


Figure 5.16: Comparison of different Gaussian widths using linear regression for the delta transition model, without applying clustering.

The baseline measurements in Figure 5.16 show that the performance increases with higher Gaussian widths up to a peak at $\sigma = 0.5$ with a running mean in the range 350-400 before declining down to around 110 for $\sigma = 5.0$. Greater values than five were not attempted due to the performance trending downwards. After running the baseline, weighted clustering was investigated using $\sigma = 0.5$ due to this value having the highest performance.

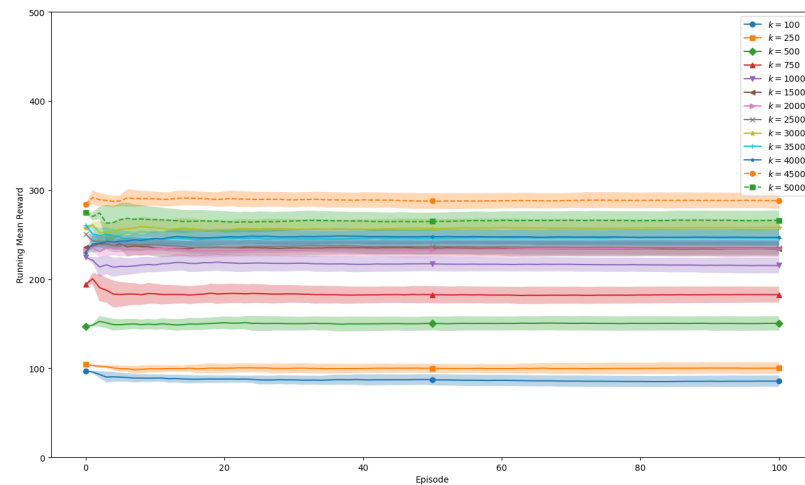


Figure 5.17: Weighted k-means clustering using $\sigma = 0.5$.

Figure 5.17 shows the results of weighted k-means clustering using $\sigma = 0.5$ for the Gaussian functions, as well as the softmax function with temperature $T = 50$ for weighting. The values of k in the range $[100, 750]$ produce running means in the range $[90, 200]$. Values $k \geq 1000$ show increasingly better performance up to a peak of around 300 running mean at $k = 4500$, before declining at $k = 5000$.

5.7 Weighted Clustering with Higher Gaussian Widths

This experiment was run using Gaussian widths of 1, 2.5, and 5.0. Weighted clustering was applied using a softmax function with temperature $T = 50$. The values of k that were tested were chosen from the same values used in the baseline to allow for a direct comparison. Additionally, the delta variation used linear regression with an error threshold of 0.01.

5.7.1 Delta Transition Model

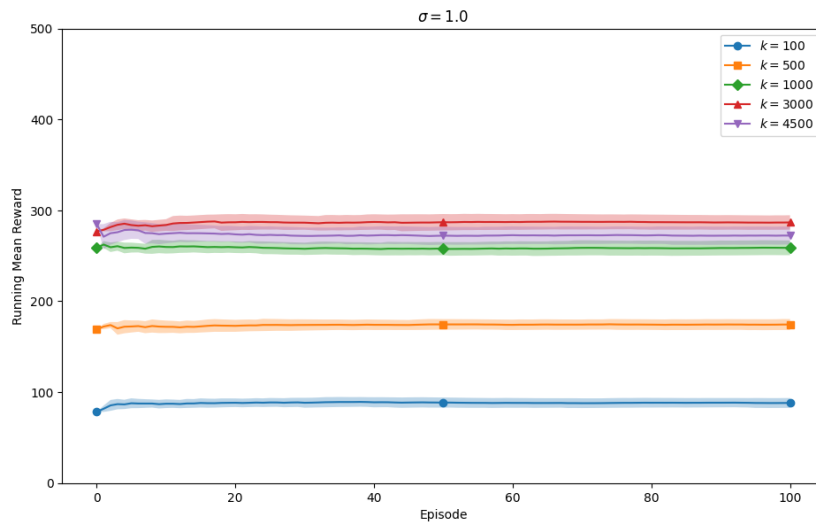


Figure 5.18: Post-clustering results for Gaussian width 1.0

Figure 5.18 shows the results for Gaussian width $\sigma = 1.0$ after weighted clustering. The figure shows a similar trend to previous results, where $k = 100$ is the lowest-performing value. The values 3000 and 4500 are similar, with $k = 3000$ showing a slightly higher running mean.

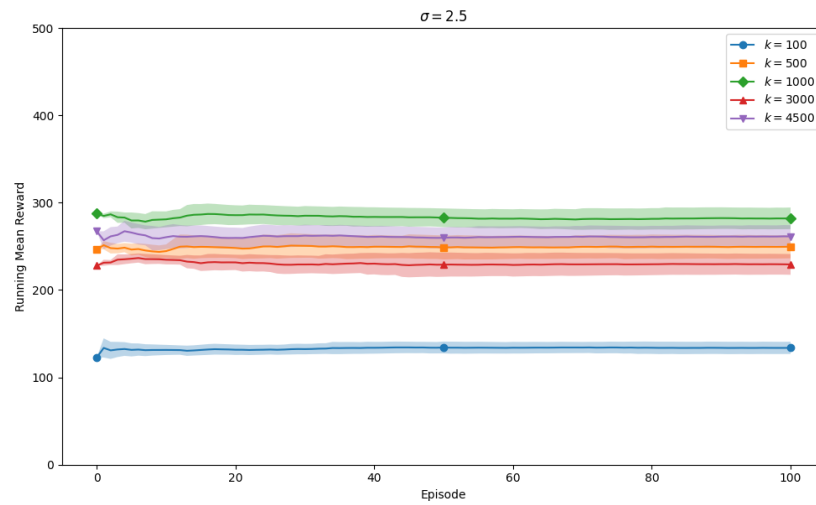


Figure 5.19: Post-clustering results for Gaussian width 2.5

For Gaussian width $\sigma = 2.5$, the results shown in Figure 5.19 are less separated, showing a higher performance for $k = 500$ than $k = 3000$. Additionally, $k = 1000$ results in the highest performance of the values tested.

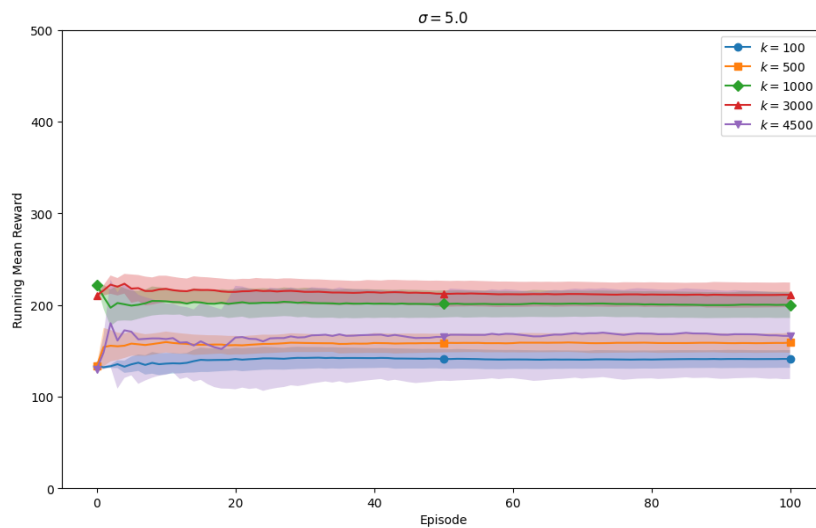


Figure 5.20: Post-clustering results for Gaussian width 5.0

Finally, Figure 5.20 shows the results of using Gaussian width $\sigma = 5.0$. Here, the results show a high standard deviation and little difference between

the values of k . The values $k = 1000$ and $k = 3000$ are close, but $k = 3000$ appears to perform marginally better.

5.7.2 Direct Transition Model

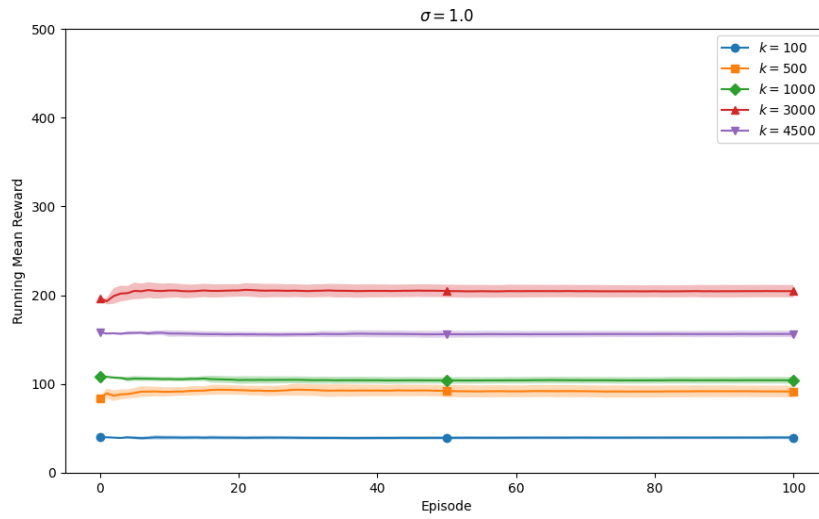


Figure 5.21: Post-clustering results for Gaussian width 1.0

Figure 5.21 shows the result of using a direct transition model with Gaussian width $\sigma = 1.0$. The plot shows that the performance increases up to a maximum of around 200 running mean at $k = 3000$ before declining at $k = 4500$.

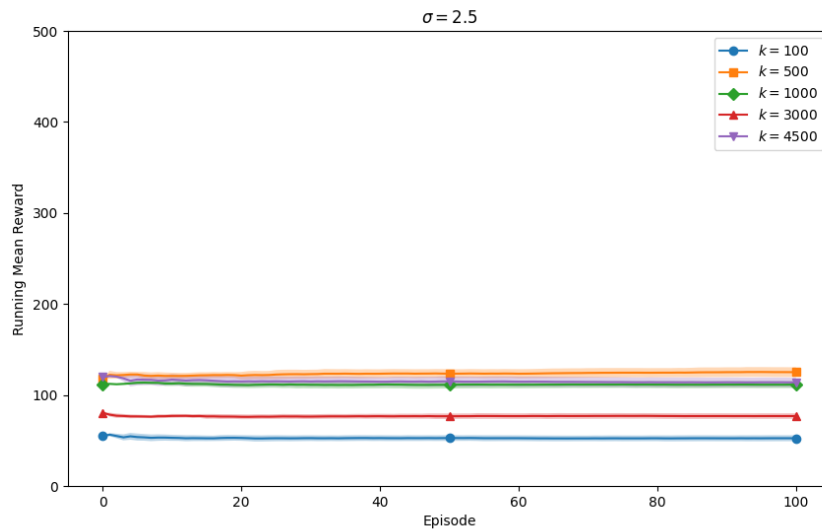


Figure 5.22: Post-clustering results for Gaussian width 2.5

Using $\sigma = 2.5$, shown in Figure 5.22, shows a lower overall performance but a peak performance at $k = 500$ instead. The runs using $k = 3000$ produced a running mean of around 80.

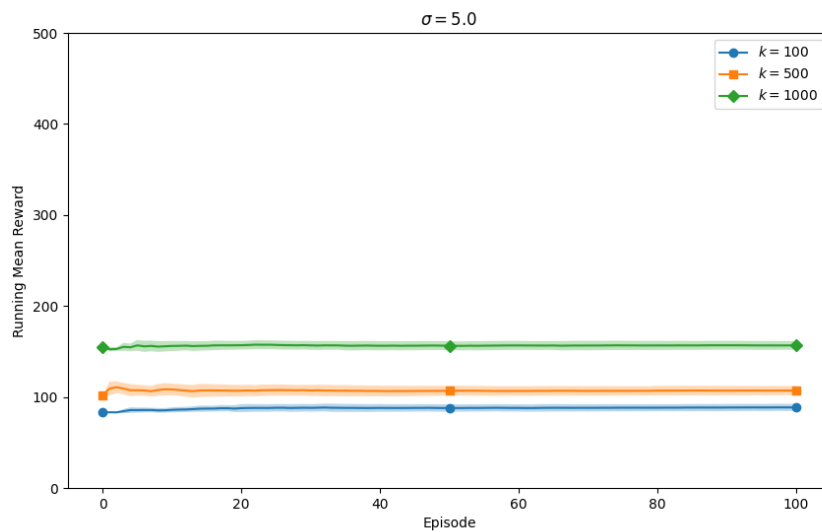


Figure 5.23: Post-clustering results for Gaussian width 5.0

Finally, Figure 5.23 shows the results of using Gaussian width $\sigma = 5.0$. For this variation, the values $k = 3000$ and $k = 4500$ are not included, as

the agent was not able to produce sufficient data to perform clustering consistently. The figure shows increasing performance, peaking at around 150-160 for $k = 1000$.

5.8 Neuron Activation States

This experiment investigated the impact of representing states using more biologically plausible methods. To limit the scope of the experiment, the results presented here were selected as they were considered representative of the trends that were observed during experimentation. The raw data of all the experiments performed are included in the appendix. For the results presented here, clustering was performed using k-means with a softmax function with temperature $T = 50$ for weighting and $k = 3000$. This experiment also used the delta transition model.

5.8.1 Gaussian Function Approach

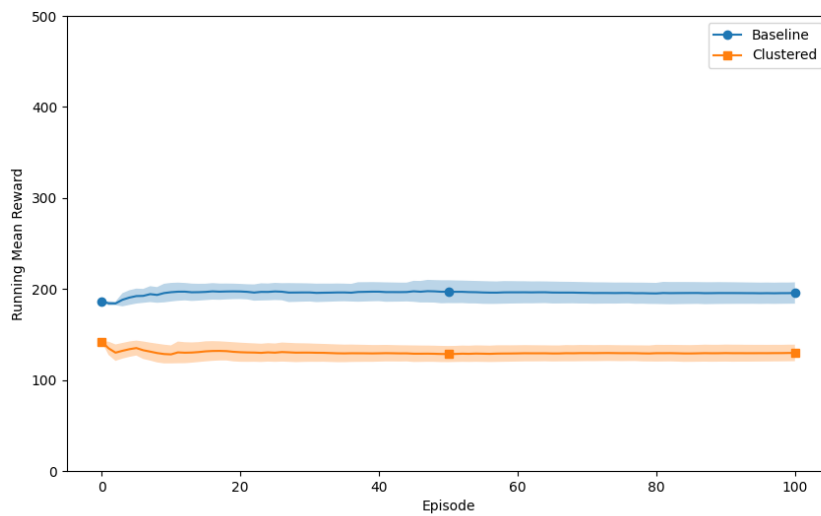


Figure 5.24: Representation of states through neuron activation using a Gaussian function.

Figure 5.24 shows the results of using a Gaussian function, with each dimension divided into 10 discrete values. The values for the interval were chosen based on average maximum and minimum values for several runs. A Gaussian width $\sigma = 0.5$ was used for both the state encoding as well as the policy and clustering functions. Variations of the experiment were performed with larger values but were too computationally intensive to allow

for reasonable data collection.

5.8.2 Positional Encoding

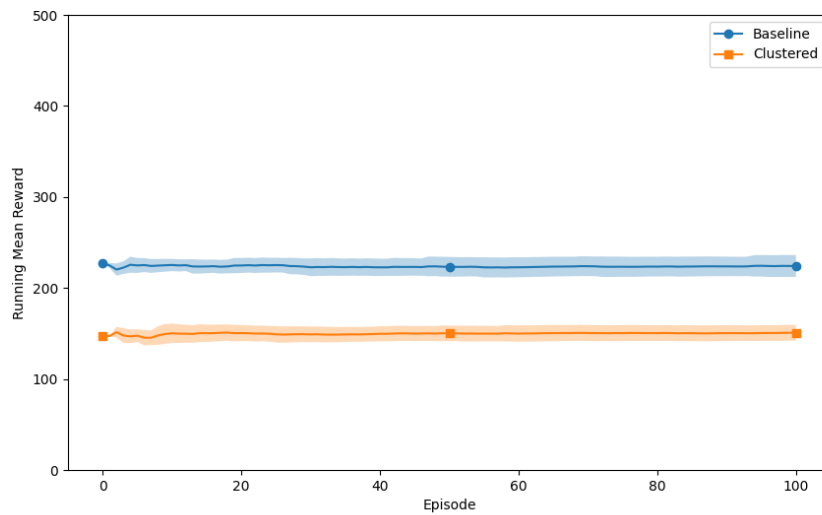


Figure 5.25: Regular positional encoding with *sin* and *cos* encodings.

The results from the first variation, shown in Figure 5.25, show a baseline reading of around 200-220, with post-clustering results of around 150. For this variation, a Gaussian width of $\sigma = 0.5$ was used for the Gaussian functions.

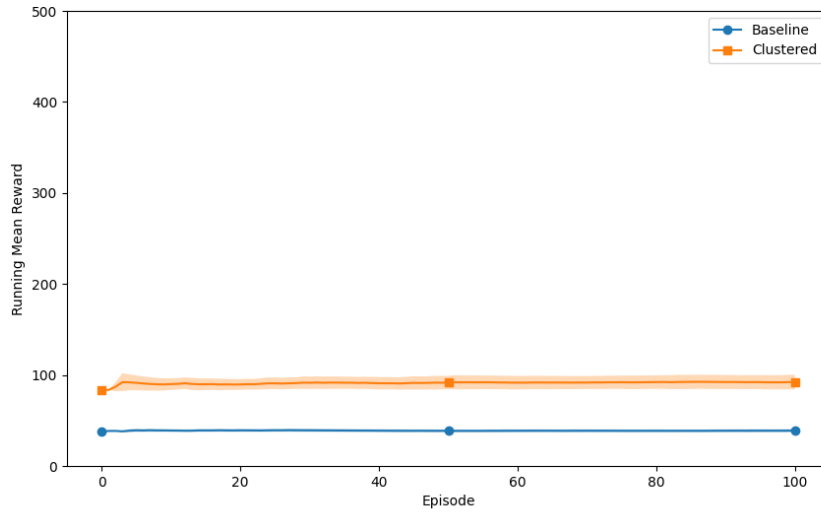


Figure 5.26: Positional encoding with cut-off for negative values.

Figure 5.26 shows the results when using a cut-off for the $\sin$ and $\cos$ functions, where negative values were replaced by zero. Due to poor performance for lower values, a Gaussian width of $\sigma = 200$ was used for this variation. For the clustering, $k = 500$ was used as lower values of k had previously been shown to improve performance for larger values of σ . The baseline reading showed a running mean of around 40-50, while post-clustering results showed a running mean of around 100.

5.9 DQN and Q-Learning

As a comparative experiment, Q-learning, DQN, and the delta variant agents were evaluated in the CartPole-v1 environment. Each agent was trained for 100 episodes across 100 independent runs to assess learning performance under constrained data availability.

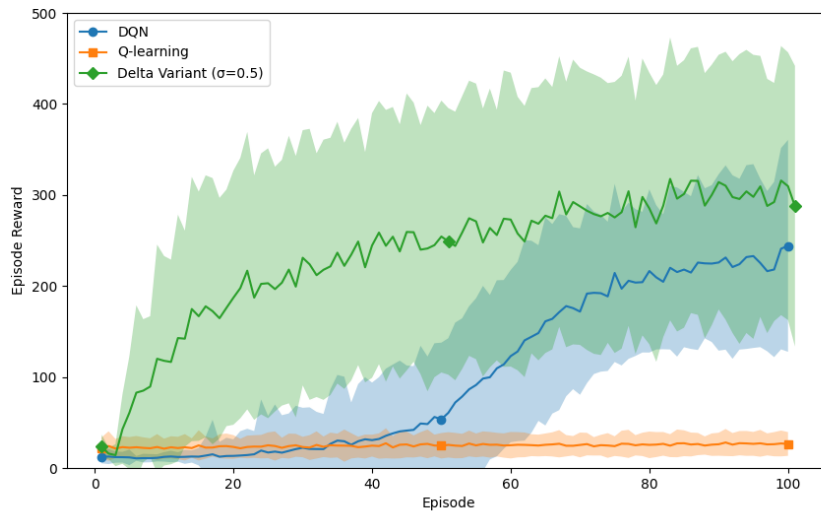


Figure 5.27: Mean episode reward and standard deviation for Q-learning, DQN, and the delta variant during training

Figure 5.27 presents the learning curves of all three agents. The delta variant (introduced in Section 5.5) outperforms both Q-learning and DQN in terms of sample efficiency, achieving higher average episode rewards within the limited training horizon. While DQN demonstrates relatively strong performance despite its model-free nature and limited data, the Q-learning agent fails to make substantial progress. These results support the theoretical discussion presented in Section 2.1.5, which posits that model-based reinforcement learning agents are generally more sample-efficient than their model-free counterparts.

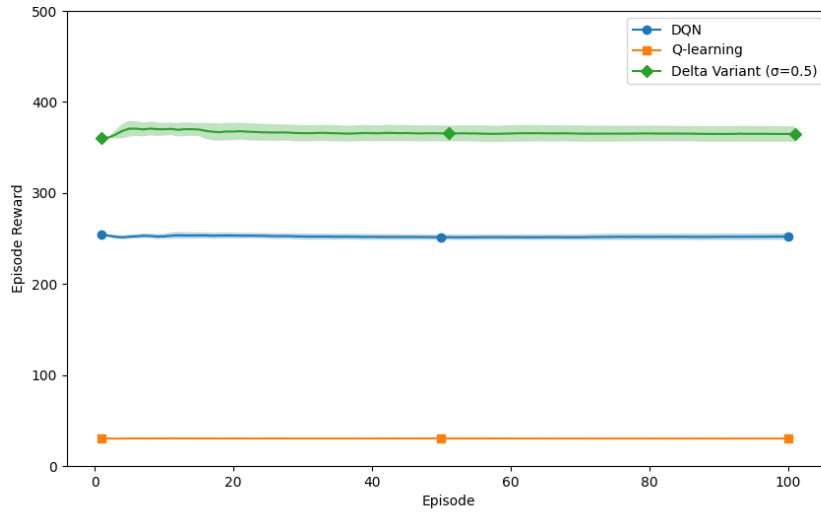


Figure 5.28: Running mean reward and standard deviation during testing of Q-learning, DQN, and delta-variant agents.

Figure 5.28 presents the running mean and standard deviation of episode rewards during the testing phase of the Q-learning, DQN, and delta variant agents. During testing, the exploration rate ϵ was set to zero for all agents to enforce purely greedy action selection, ensuring that each agent followed its learned policy without stochastic exploration. The results indicate that the delta variant substantially outperforms Q-learning and consistently achieves a running mean reward approximately 100 points higher than DQN under the constraints of limited data availability.

6. Discussion

This chapter will reflect on the results presented in the results chapter and discuss different factors that may have had an impact on the results. Each experiment will be discussed separately. To better analyse the impact of individual hyperparameters, it was deemed beneficial to lock down specific values for each of the broader parameters early on. This allowed the study to constrain further experiments, which in turn made it easier to highlight the results of each individual experiment.

6.1 Direct Transition Model without Clustering

The initial experiment provided a baseline for future comparisons using the initial version of the transition model. In this experiment, the impact of the Gaussian width was investigated and was found to have a very large effect on the agent's results. The highest recorded average was from $\sigma = 0.5$, so this value was used for further experiments, assuming that it would produce the best results. Higher values produced progressively worse averages, with $\sigma = 5.0$ producing the lowest recorded running mean reward. Due to this trend, greater values were not investigated thoroughly in this study.

The t-SNE plot from figure 5.2 showed that similar states largely produce similar rewards. This was significant due to the impact this assumption had on the policy of the agent, where the previously achieved reward of states was used to calculate the utility of later states. Confirming this assumption meant that the agent could reliably calculate utilities based on previous experience and make informed decisions based on these calculations.

6.2 Direct Transition Model with Regular K-Means

Clustering without any weighting showed poor performance compared to the baseline with no clustering. At a Gaussian width of 0.5, the previous experiment showed a running mean of just over 300, while regular k-means at most reached a mean of just under 250. The difference can be seen in figure 5.5. The hypothesis that compressing the agent's model would preserve or improve performance was therefore falsified and showed that

further experimentation was needed. Additionally, the values of k were still considered quite large for this study, which primarily was focused on investigating larger amounts of compression.

Interestingly, there appears to be a range of values for k where performance peaks. Values outside of this range lead to worse performance. Although the value 3000 was chosen for further experimentation, there was an indication that tuning other parameters or testing other methods could allow smaller values of k .

These results are likely a consequence of the lack of any form of relation between the clustering process and the rewards earned. In biological processes, there is typically some sort of feedback that signifies whether an event or experience is worth remembering, which is not reflected when using regular k-means. Since reinforcement learning is based on exploration and rewards, this form of clustering likely removes too much information without compensating for it in another way. As a result, the agent does not improve its performance.

6.3 Weighted K-Means

The initial weighting of using a power function produced satisfactory results, but the method was considered to have significant weaknesses compared to other methods. Due to the unbounded range of the output, overflow could, in theory, be possible for very large reward values. Additionally, the lack of normalisation would make it difficult to maintain consistency across different reward scales. For these reasons, alternatives to the power function were more desirable.

Variations using the softmax function produced the best results of the experiment, with a temperature parameter of $T = 50$ producing the highest running mean of the tested values. As mentioned above, the agent was assumed to perform better with a greater number of high-reward states. For this reason, it was initially assumed that a lower temperature would be desirable, as this would push the weights of low-reward states closer to zero. This turned out not to be the case, with higher temperatures greatly outperforming lower temperatures. These results may indicate that although high-reward states are important for the agent's performance, low-reward states are still required for the agent to perform well. There may also be an interaction between the Gaussian width used for utility calculations and the distribution of states, meaning that having accurate representations of low-reward states is better than having only high-reward states.

The final variation used the logarithm of the softmax function and shifted these values to new upper bounds before exponentiating. This was done due to concerns about numerical stability when using regular softmax, but

from the results presented, this did not appear to be an issue. Using this method produced a lower running mean than the other methods. This may have been a consequence of the same effect described above, where the excessive numerical values may have resulted in a poorer distribution of states than desired.

The overall results of this experiment showed a marked improvement when using weighted k-means over regular k-means, as visualised in figure 5.10. This aligns with the assumption that an agent with a greater number of accurate representations of high-reward states would outperform one with fewer such states. These results also support the hypothesis that weighted clustering would improve performance over unweighted clustering. Despite this, none of the variations of weighted clustering performed better than the baseline without any clustering.

6.4 Direct Transition Model using Online Clustering

Both online clustering and the Gaussian Mixture Model were tested to investigate different methods of vector quantisation. Overall, neither of the methods were used for further experimentation.

6.4.1 Online Clustering

Overall, the experiment using online clustering yielded poor results, even after the agent was allowed to continue training after clustering. The greatest recorded running mean of the variations was only around 80, which is far below both regular and weighted k-means using the same hyperparameters. Different Gaussian widths were tested for clustering and policy based on initial indications that this configuration might improve outcomes, but no significant improvements were observed.

It is not entirely clear why the agent performs worse after being allowed to continue training. There may have been issues with the implementation of the clustering algorithm, but despite this, one would expect to see an improvement after five additional training sessions of 100 episodes. This result may be related to the previously mentioned effect of the agent focusing too much on high-reward states. Although some improvement was seen when applying a filter of 0.5 to the Gaussian function during the reward calculation for the centroids, this improvement was not sufficient to produce satisfactory results. For these reasons, online clustering was not considered for future experiments.

6.4.2 Gaussian Mixture Model

While k-means clustering produces spherical (isotropic) clusters due to its reliance on Euclidean distances, the Gaussian mixture model (GMM) is capable of modelling elliptical clusters by incorporating full covariance matrices. This increased flexibility can result in more accurate representations of the underlying data distributions. Although GMM is inherently a soft clustering algorithm, assigning each data point a probability of belonging to each cluster, the implementation in this experiment used hard assignments via the `predict()` method, which assigns each point to the component with the highest posterior probability. As a result, the clustering behaviour becomes more similar to that of k-means.

This similarity in assignment may partially explain why GMM and weighted k-means achieved comparable performance. Notably, GMM matched the performance of the weighted k-means setup using temperature $T = 50$ despite GMM being used without any state weighting or additional hyperparameter tuning beyond the previously established optimal values. This suggests that GMM may have latent potential for even higher performance if combined with weighting strategies.

However, due to the high computational cost associated with fitting GMMs, especially at large values of $n_{\text{components}}$, the scope of experimentation was limited. As a result, further exploration of GMM under different configurations (e.g., with weighted states or alternative covariance structures) remains an open direction for future work.

6.5 Delta Transition Model using Splines

Comparing the graphs for the Gaussian widths of Experiments 5.1 and 5.5, a consistent trend can be observed: Gaussian widths in the range $\sigma \in [0.3, 0.5]$ yield the highest running mean rewards, while both lower ($\sigma < 0.3$) and higher ($\sigma > 0.5$) values lead to performance degradation. Interestingly, the delta variant achieves its best performance at $\sigma = 0.3$, whereas the direct transition model performs best with $\sigma = 0.5$. This discrepancy may be attributed to differences in how similarity is computed between transitions. Because the delta variant's prediction model yields a more accurate estimate of the next state, the Gaussian function better captures the true similarity between transitions. Consequently, lower values of σ - which emphasise closer neighbours more strongly - are more effective for the delta variant, enabling a more precise representation of the underlying transition dynamics.

Moreover, the two Figures 5.1 and 5.13 show that the delta variant consistently outperforms the direct transition model baseline, suggesting that

its architecture provides more accurate state transition estimates and thus leads to improved policy learning.

As illustrated in Figure 5.14, the model achieves its highest performance when the error threshold ϵ is smallest. A lower error threshold causes the spline-based prediction model to update more frequently, as the difference between the predicted and actual deltas more often exceeds the specified threshold. This leads to a more accurate transition model, which in turn improves the agent's ability to learn effective policies. Interestingly, the results show that $\epsilon = 10^{-2}$ and $\epsilon = 10^{-4}$ yield similar performance, while $\epsilon = 10^{-3}$ slightly outperforms $\epsilon = 10^{-4}$. This non-monotonic pattern was not anticipated, and the underlying cause remains unclear. Although such variations are often attributed to stochasticity in the learning process, the consistency across 100 runs suggests that randomness alone may not fully explain the outcome. Consultation with the research supervisor did not produce a definitive explanation, indicating that further investigation is required to determine the source of this behaviour, which could involve deeper interactions between the prediction model, learning dynamics, and data distribution. In summary, these results support the hypothesis that modelling transitions as a delta would improve performance.

Figures 5.3 and 5.15 illustrate differing trends in running mean reward for the delta variant and direct transition models. The delta variant exhibits a non-linear performance profile, achieving local maxima at $K = 2000$ and $K = 3500$, whereas the delta transition model demonstrates an approximately linear increase in performance up to $K = 4500$, beyond which the reward begins to decline. The underlying cause of this divergence is unclear; however, it is conceivable that the direct transition model might display a similar non-linear pattern with additional local optima if tested with larger values of K . The delta variant achieves higher performance with fewer clusters, suggesting it is more effective than the direct transition model in data-scarce settings, likely due to its better representation of the underlying transition dynamics discussed in the first paragraph of this subsection.

6.6 Delta Transition Model using Linear Regression

This experiment showed that using linear regression in the transition model performed nearly identically to the variation using splines, with greater computational efficiency. Since the previous experiment showed that the error threshold ϵ provided little in terms of performance, this experiment was run using the threshold $\epsilon = 0.01$.

Comparing the baseline in figure 5.16 with the baseline from the previous experiment in figure 5.13, there is little difference in the running means for the different values of σ . A noteworthy difference is that linear regression has higher performance for $\sigma = 0.5$ than $\sigma = 0.3$, while the opposite is true for splines. This may be a result of randomness in the experimental data, but it may also be that linear regression produces a slightly less accurate transition model.

Applying weighted clustering using this transition model shows a similar trend to previous experiments. Performance increases as k increases up to a peak running mean at $k = 4500$. Larger values than this show a decline in performance. With these results in mind, it's clear that using a delta transition model is better in terms of performance but does not seem to have much of an effect on the clustering using the current hyperparameters. The results of this experiment support the hypothesis that linear regression produces results similar to those of splines while reducing computational overhead.

6.7 Weighted Clustering with Higher Gaussian Widths

The previous experiments showed that the performance of the agent declined considerably when using a higher σ for the Gaussian functions. For this experiment, these values were revisited to investigate the impact of weighted clustering on these cases. Both the variations using delta and direct transition models were tested and compared to the baselines previously established. Weighting was performed using the softmax function with temperature $T = 50$.

6.7.1 Delta Transition Model

Comparing figures 5.18 and 5.16, there does not seem to be any improvement from weighted clustering. The baseline readings for Gaussian width $\sigma = 1.0$ show a running mean of around 310, while the highest running mean after clustering is around 290-300. This is better than previous results, but the post-clustering results do not outperform the baseline.

Looking at figure 5.19, however, there is a clear improvement in performance after clustering compared to the baseline. While the baseline for Gaussian width $\sigma = 2.5$ averages around 190-200, the results after clustering show averages up to 300 at $k = 1000$. Although $k = 100$ still decreases the performance, the runs using $k = 500$ show averages comparable to higher values such as $k = 4500$.

Using a higher Gaussian width of $\sigma = 5.0$, the agent's performance appears to decline again. Notably, the results at this value are much more clumped

together and show a greater standard deviation. Although $k = 3000$ produces the highest average, the difference between $k = 500$ and $k = 4500$ is small.

6.7.2 Direct Transition Model

Looking at the runs using Gaussian width $\sigma = 1.0$ in figure 5.1, and comparing to figure 5.21, there does not appear to be an improvement in this case either. The highest achieved running mean is at $k = 3000$ before declining again for larger values.

This also appears to be the case when looking at figure 5.22, which also shows a lower performance when compared to the baseline in figure 5.1. However, $k = 500$ appears to perform relatively better, producing the highest running mean of the values tested.

Looking at figure 5.23, however, the performance increases after clustering even for small values of k . A noteworthy observation here is that even the runs using $k = 100$ produced a running mean higher than the baseline, although marginally so. This observation could, however, be due to the overall poor performance at this Gaussian width, which led to fewer overall observations in the model.

6.7.3 Summary

This experiment indicates that using a higher value of σ for the Gaussian functions may lead to an improvement in performance after clustering compared to the baseline, which supports the initial hypothesis. A reason for this could be that smaller values require more precise data for the policy to make informed choices, while larger values work better for the estimates produced by weighted k-means clustering. Although there is a relative improvement in performance, it should be pointed out that these parameters still do not outperform the baseline at $\sigma = 0.5$. However, due to factors encountered in real-life applications, such as noise in the sensors, there may be practical benefits to using larger values for the Gaussian functions.

6.8 Neuron Activation States

Overall, the results from this experiment did not show any improvement in using neuron activation representations of states. Additionally, the experiments were computationally intensive, making data gathering and experimentation quite slow.

The first experiment was performed using a Gaussian function to calculate the activations for the states. Although the performance was poorer than

previous experiments using the standard states from the Cart Pole environment, the agent retained some decision-making ability, albeit with a lower running mean than before. This is in line with previous experiments where lower values of the Gaussian width led to worse performance after clustering.

Positional encoding performed somewhat better than the Gaussian function approach but did not outperform earlier methods. Again, the performance post-clustering was poorer but still showed that the agent retained some decision-making ability.

Lastly, the positional encoding approach with a cut-off performed significantly worse than previous variations. Due to the poor performance of the lower values of σ , a much higher value was needed to produce any useful data. Despite the poor performance, it is noteworthy that the clustered version performs better than the baseline. This further supports the earlier findings that larger values of Gaussian width generally lead to better performance after clustering when compared to the baseline.

In summary, the hypothesis that more biologically plausible representations of states would improve performance could not be supported. It should be noted that this approach was not studied in great detail, and although the initial results show poor performance, there may be some combination of hyperparameters that is still viable.

6.9 DQN and Q-Learning

As a comparative benchmark, Deep Q-Network (DQN) and Q-learning were trained and evaluated on CartPole-v1. The hypothesis from Section **IX.** stated that model-free agents would underperform relative to model-based variants under low-sample conditions, with Q-learning expected to struggle due to the explosion of discrete states and its reliance on extensive exploration. The goal was to provide reference points for performance under the same constraints as our other model-based variants. Both agents were limited to 100 training episodes.

The DQN achieves a running mean of around 250, which is significantly worse than the baseline of the model-based agent using a delta transition model. Figures 5.27 and 5.28 show that the delta variant outperforms both Q-learning and DQN during both the training and testing phases. In fact, Figure 5.27 shows that the delta variant learns dramatically quicker than the others as well.

This experiment supports the conclusion that while model-free agents such as DQN are capable of performing well under limiting training budgets, they are still outperformed in sample efficiency by model-based variants.

The results further show the limitations of Q-learning methods in high-dimensional environments, particularly when exploration opportunities are limited.

Under identical training conditions, the delta transition model consistently surpasses both DQN and Q-learning, achieving running mean rewards roughly 100 points higher than DQN during testing. These findings corroborate theoretical expectations (Section 2.1.5) that model-based reinforcement learning methods are more sample-efficient.

7. Conclusion and Future Work

This chapter will summarise the findings of the experiments performed and the discussion from chapter 6. Additionally, the impacts of the study will be briefly discussed, relating back to the sustainability aspects covered in Section 2.8. Finally, future work for the study will be discussed, particularly focusing on the experiments that were not possible due to time constraints.

7.1 Findings

Initial experiments showed that the agent was able to achieve good performance after 100 episodes. This was in line with the theory covered in Section 2.1.5.

The main difference in this particular approach to model-based reinforcement learning (MBRL) was the wide use of Gaussian functions as covered in Section 2.2.2. Through experimentation, it was found that the Gaussian width σ for these functions had a major impact on performance, and it was early on made clear that this would be a point of interest in all the experiments. Although the earliest experiments found that the values in the range $\sigma = [0.3, 0.5]$ produced the best overall performance, larger values like $\sigma = 2.5$ enabled better performance post-clustering compared to before. This improvement, however, came at the cost of overall performance, as the agent still was not able to outperform the best baseline results for lower values of σ .

Clustering was the main approach to biological plausibility in this study. In general, applying clustering to an agent reduced the performance significantly. This was also the case after applying weighting, although the performance was improved compared to unweighted approaches. Additionally, lower values of k were found to be substantially worse than larger ones. However, as mentioned above, it was observed that using larger values of the Gaussian width σ led to an improvement in performance after clustering compared to the baseline. This observation could indicate that weighted clustering is advantageous in partially observable environments, as a larger Gaussian width better accounts for imprecise data.

Finally, experimentation showed that representing states through neuron activation approaches did not yield any improvements in performance. This

was first tested using Gaussian functions, then using more conventional positional encoding methods. It should be mentioned, however, that the methods tested here had a limited scope. Other methods than the ones tested in this study may still yield improvements.

7.2 Impact

This study demonstrated, among other findings, that model-based reinforcement learning is more sample-efficient than model-free approaches. Sample efficiency has sustainability implications, particularly when an agent must interact with a physical environment. Even for more conventional environments, the reduced computational overhead reduces the overall emissions from the training process. Although the clustering methods used in the study did not improve performance for all cases, the cases where performance was increased could include physical applications. However, this was not investigated in this study.

7.3 Future work

Future work on the project could focus on more sophisticated clustering algorithms. In particular, the Deep Embedded Clustering algorithm developed by Xie, Girshick and Farhadi [79] was considered for experimentation. However, time did not allow for the implementation of this algorithm, so it remains to be seen how this will impact the agent.

Another potential avenue for future work is the incorporation of reward shaping techniques. As mentioned in Section 4.2.1, rewards were not discounted in this study. However, as discussed in Section 2.1.3, discounting future rewards is a standard practice in reinforcement learning to emphasize long-term gains. In addition, shaping the reward function to provide more informative or dense feedback during training could significantly improve the agent's sample efficiency, convergence rate, and overall learning stability.

An advantage of the model-based reinforcement learning agent implemented in this study is that the data collected by one such agent can also be used by another. This entails that several agents could be trained simultaneously, allowing the training to be done in parallel. It may be interesting to investigate the difference in performance between an agent collecting its data in a single run and several agents collecting data in parallel.

Furthermore, as a result of the discussion in chapter 6, it was hypothesised that a more even dispersion of the cluster centroids could improve performance. To achieve this, future studies could test other clustering algorithms or weighting functions for the algorithms covered here.

Finally, the agent could be tested in partially observable environments. As discussed in Section 6.5, higher values of the Gaussian width σ showed promise for post-clustering performance. From this, it was hypothesised that the agent might benefit from clustering in environments where the data observed contains noise. The agent could, for example, be tested in simple robotic implementations to test this hypothesis.

Bibliography

- [1] M. Ahmed, R. Seraj and S. M. S. Islam, 'The k-means algorithm: A comprehensive survey and performance evaluation,' *Electronics*, vol. 9, no. 8, 2020, ISSN: 2079-9292. DOI: 10.3390/electronics9081295. [Online]. Available: <https://www.mdpi.com/2079-9292/9/8/1295>.
- [2] A. Banerjee, D. Ghosh and S. Das, 'Evolving network topology in policy gradient reinforcement learning algorithms,' in *2019 Second International Conference on Advanced Computational and Communication Paradigms (ICACCP)*, IEEE, Feb. 2019, pp. 1–5. DOI: 10.1109/ICACCP.2019.8882916.
- [3] A. G. Barto, R. S. Sutton and C. W. Anderson, 'Neuronlike adaptive elements that can solve difficult learning control problems,' *IEEE Transactions on Systems, Man, and Cybernetics*, vol. SMC-13, no. 5, pp. 834–846, 1983. DOI: 10.1109/TSMC.1983.6313077.
- [4] R. Bellman, *Dynamic programming*, eng. Princeton, N.J: Princeton University Press, 1957.
- [5] Y. Bengio, D. Lee, J. Bornschein and Z. Lin, 'Towards biologically plausible deep learning,' *CoRR*, vol. abs/1502.04156, 2015. arXiv: 1502.04156. [Online]. Available: <http://arxiv.org/abs/1502.04156>.
- [6] J. R. Bertram and P. Wei, 'Explainable deterministic mdps,' *CoRR*, vol. abs/1806.03492, 2018. arXiv: 1806.03492. [Online]. Available: <http://arxiv.org/abs/1806.03492>.
- [7] Boston Dynamics, *Starting on the right foot with reinforcement learning*, <https://bostondynamics.com/blog/starting-on-the-right-foot-with-reinforcement-learning>, <https://bostondynamics.com/blog/starting-on-the-right-foot-with-reinforcement-learning>, 2024.
- [8] C. M. Bowyer. 'Setting up the lunar lander reinforcement learning (rl) environment.' Accessed: Apr. 4, 2025. (Oct. 2022), [Online]. Available: <https://medium.com/@CalebMBowyer/setting-up-the-lunar-lander-reinforcement-learning-rl-environment-29d285aa9508>.

- [9] E. Candela, O. Doustaly, L. Parada, F. Feng, Y. Demiris and P. Angeloudis, 'Risk-aware controller for autonomous vehicles using model-based collision prediction and reinforcement learning,' *Artificial Intelligence*, vol. 320, p. 103 923, 2023, ISSN: 0004-3702. DOI: <https://doi.org/10.1016/j.artint.2023.103923>. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0004370223000693>.
- [10] M.-A. Chadi and H. Mousannif, 'Understanding reinforcement learning algorithms: The progress from basic q-learning to proximal policy optimization,' *arXiv preprint*, 2023. arXiv: 2304.00026 [cs.LG]. [Online]. Available: <https://arxiv.org/abs/2304.00026>.
- [11] D. Chong Yi Wu, 2024. [Online]. Available: <https://fse.studenttheses.ub.rug.nl/33922/1/thesiss4752244.pdf>.
- [12] K. Chua, R. Calandra, R. McAllister and S. Levine, *Deep reinforcement learning in a handful of trials using probabilistic dynamics models*, 2018. arXiv: 1805.12114 [cs.LG]. [Online]. Available: <https://arxiv.org/abs/1805.12114>.
- [13] O. W. in Data team, 'Ensure sustainable consumption and production patterns,' *Our World in Data*, 2023, <https://ourworldindata.org/sdgs/responsible-consumption-production>.
- [14] O. W. in Data team, 'Take urgent action to combat climate change and its impacts,' *Our World in Data*, 2023, <https://ourworldindata.org/sdgs/climate-action>.
- [15] O. C. Eidheim, *Online arbitrary shaped clustering through correlated gaussian functions*, 2023. arXiv: 2302.06335 [cs.LG]. [Online]. Available: <https://arxiv.org/abs/2302.06335>.
- [16] Farama Foundation, *Cart pole - gymnasium documentation*, Online, Accessed: Apr. 7, 2025. [Online]. Available: https://gymnasium.farama.org/environments/classic_control/cart_pole/.
- [17] Farama Foundation, *Lunar lander - gymnasium documentation*, Online, Accessed: Apr. 7, 2025. [Online]. Available: https://gymnasium.farama.org/environments/classic_control/cart_pole/.

- [18] M. S. Floater, *An introduction to spline theory*, Lecture Notes, University of Oslo, Draft, May 22, 2023, 2023.
- [19] S. Gadgil, Y. Xin and C. Xu, *Solving the lunar lander problem under uncertainty using reinforcement learning*, arXiv preprint arXiv:2011.11850, Accessed: Apr. 4, 2025., 2020. [Online]. Available: <https://arxiv.org/abs/2011.11850>.
- [20] C. Gao and D. Wang, 'Comparative study of model-based and model-free reinforcement learning control performance in hvac systems,' *Journal of Building Engineering*, vol. 74, p. 106852, 2023, ISSN: 2352-7102. DOI: <https://doi.org/10.1016/j.jobe.2023.106852>. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2352710223010318>.
- [21] J. Gehring, M. Auli, D. Grangier, D. Yarats and Y. N. Dauphin, 'Convolutional sequence to sequence learning,' in *Proceedings of the 34th International Conference on Machine Learning*, D. Precup and Y. W. Teh, Eds., ser. Proceedings of Machine Learning Research, vol. 70, PMLR, Jun. 2017, pp. 1243–1252. [Online]. Available: <https://proceedings.mlr.press/v70/gehring17a.html>.
- [22] M. Ghasemi and D. Ebrahimi, 'Introduction to reinforcement learning,' *arXiv preprint arXiv:2408.07712*, Dec. 2024, Department of Computer Science, Wilfrid Laurier University, Waterloo, Canada. [Online]. Available: <https://arxiv.org/abs/2408.07712>.
- [23] I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*. Cambridge, MA: MIT Press, 2016, ISBN: 978-0262035613.
- [24] R. Gray, 'Vector quantization,' *IEEE ASSP Magazine*, vol. 1, no. 2, pp. 4–29, 1984. DOI: 10.1109/MASSP.1984.1162229.
- [25] H. Guo, 'A simple algorithm for fitting a gaussian function [dsp tips and tricks],' *IEEE Signal Processing Magazine*, vol. 28, no. 5, pp. 134–137, 2011. DOI: 10.1109/MSP.2011.941846.
- [26] T. Haarnoja, A. Zhou, P. Abbeel and S. Levine, 'Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor,' *arXiv preprint*, 2018. arXiv: 1801.01290 [cs.LG]. [Online]. Available: <https://arxiv.org/abs/1801.01290>.

- [27] T. Haarnoja, A. Zhou, K. Hartikainen, G. Tucker, S. Ha, J. Tan, V. Kumar, H. Zhu, A. Gupta, P. Abbeel and S. Levine, 'Soft actor-critic algorithms and applications,' *arXiv preprint*, 2018. arXiv: 1812.05905 [cs.LG]. [Online]. Available: <https://arxiv.org/abs/1812.05905>.
- [28] D. Hafner, T. Lillicrap, J. Ba and M. Norouzi, *Dream to control: Learning behaviors by latent imagination*, 2020. arXiv: 1912.01603 [cs.LG]. [Online]. Available: <https://arxiv.org/abs/1912.01603>.
- [29] D. Hafner, T. Lillicrap, I. Fischer, R. Villegas, D. Ha, H. Lee and J. Davidson, *Learning latent dynamics for planning from pixels*, 2019. arXiv: 1811.04551 [cs.LG]. [Online]. Available: <https://arxiv.org/abs/1811.04551>.
- [30] N. Hansen, X. Wang and H. Su, *Temporal difference learning for model predictive control*, 2022. arXiv: 2203.04955 [cs.LG]. [Online]. Available: <https://arxiv.org/abs/2203.04955>.
- [31] M. Hessel, J. Modayil, H. van Hasselt, T. Schaul, G. Ostrovski, W. Dabney, D. Horgan, B. Piot, M. Azar and D. Silver, 'Rainbow: Combining improvements in deep reinforcement learning,' *arXiv preprint arXiv:1710.02298*, 2017.
- [32] T. Hester and P. Stone, 'Texlore: Real-time sample-efficient reinforcement learning for robots,' *Machine Learning*, vol. 90, no. 3, pp. 385–429, Oct. 2012. DOI: 10.1007/s10994-012-5322-7. [Online]. Available: <https://doi.org/10.1007/s10994-012-5322-7>.
- [33] G. E. Hinton and S. Roweis, 'Stochastic neighbor embedding,' in *Advances in Neural Information Processing Systems*, S. Becker, S. Thrun and K. Obermayer, Eds., vol. 15, MIT Press, 2002. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2002/file/6150ccc6069bea6b5716254057a194ef-Paper.pdf.
- [34] B. Hoffman, J. Cheng, C. Li and S. Coros, *Learning more with less: Sample efficient dynamics learning and model-based rl for locomanipulation*, 2025. arXiv: 2501.10499 [cs.RD]. [Online]. Available: <https://arxiv.org/abs/2501.10499>.
- [35] M. S. Holubar and M. A. Wiering, 'Continuous-action reinforcement learning for playing racing games: Comparing spg to ppo,' *arXiv pre-*

- print*, 2020. arXiv: 2001.05270 [cs.LG]. [Online]. Available: <https://arxiv.org/abs/2001.05270>.
- [36] M. M. Ibrahim, *A comparative study of sac and ppo algorithms in reservoir water level control simulation*, Master's final report, University of Sheffield, Accessed: 2024-11-16, 2020. [Online]. Available: https://github.com/Muzhaffar99/A-Comparative-Study-of-SAC-and-PP0/blob/main/MSc_Final%20report_Muzhaffar%20Maruf%20Ibrahim.pdf.
- [37] M. Janner, J. Fu, M. Zhang and S. Levine, *When to trust your model: Model-based policy optimization*, 2021. arXiv: 1906.08253 [cs.LG]. [Online]. Available: <https://arxiv.org/abs/1906.08253>.
- [38] Y.-X. Jin, Z.-W. Wang, H.-Z. Xu, W.-F. Zhuang, M.-J. Hu and D. E. Liu, *Ppo-q: Proximal policy optimization with parametrized quantum policies or values*, 2025. arXiv: 2501.07085 [quant-ph]. [Online]. Available: <https://arxiv.org/abs/2501.07085>.
- [39] L. P. Kaelbling, M. L. Littman and A. W. Moore, 'Reinforcement learning: A survey,' *Journal of Artificial Intelligence Research*, vol. 4, pp. 237–285, May 1996. DOI: 10.1613/jair.301. [Online]. Available: <https://www.jair.org/index.php/jair/article/view/10166>.
- [40] Ł. Kaiser, M. Babaeizadeh, P. Miłoś, B. Osiński, R. H. Campbell, K. Czechowski, D. Erhan, C. Finn, P. Kozakowski, S. Levine, A. Mohiuddin, R. Sepassi, G. Tucker and H. Michalewski, 'Model-based reinforcement learning for atari,' *arXiv preprint arXiv:1903.00374v5*, 2024, Version 5, April 3, 2024.
- [41] E. Kloberdanz, K. G. Kloberdanz and W. Le, 'Deepstability: A study of unstable numerical methods and their solutions in deep learning,' in *Proceedings of the 44th International Conference on Software Engineering*, ser. ICSE '22, Pittsburgh, Pennsylvania: Association for Computing Machinery, 2022, pp. 586–597, ISBN: 9781450392211. DOI: 10.1145/3510003.3510095. [Online]. Available: <https://doi.org/10.1145/3510003.3510095>.
- [42] M. Kölle, T. Witter, T. Rohe, G. Stenzel, P. Altmann and T. Gabor, *A study on optimization techniques for variational quantum circuits*

- in reinforcement learning*, 2024. arXiv: 2405.12354 [quant-ph]. [Online]. Available: <https://arxiv.org/abs/2405.12354>.
- [43] I. Kostrikov, D. Yarats and R. Fergus, *Image augmentation is all you need: Regularizing deep reinforcement learning from pixels*, 2021. arXiv: 2004.13649 [cs.LG]. [Online]. Available: <https://arxiv.org/abs/2004.13649>.
- [44] A. Kumar, A. Zhou, G. Tucker and S. Levine, 'Conservative q-learning for offline reinforcement learning,' version v3, *arXiv preprint*, Aug. 2020. arXiv: 2006.04779 [cs.LG]. [Online]. Available: <https://arxiv.org/abs/2006.04779>.
- [45] S. Kumar, 'Balancing a cartpole system with reinforcement learning - a tutorial,' *arXiv preprint*, 2020. arXiv: 2006.04938 [cs.R0]. [Online]. Available: <https://arxiv.org/abs/2006.04938>.
- [46] T. Kurutach, I. Clavera, Y. Duan, A. Tamar and P. Abbeel, *Model-ensemble trust-region policy optimization*, 2018. arXiv: 1802.10592 [cs.LG]. [Online]. Available: <https://arxiv.org/abs/1802.10592>.
- [47] Y. Lu, M. S. Squillante and C. W. Wu, *A control-model-based approach for reinforcement learning*, Accessed: Apr. 4, 2025, 2019. arXiv: 1905.12009 [cs.LG]. [Online]. Available: <https://arxiv.org/abs/1905.12009>.
- [48] L. van der Maaten and G. Hinton, 'Visualizing data using t-sne,' *Journal of Machine Learning Research*, vol. 9, no. 86, pp. 2579–2605, 2008. [Online]. Available: <http://jmlr.org/papers/v9/vandermaaten08a.html>.
- [49] V. Micheli, E. Alonso and F. Fleuret, 'Transformers are sample-efficient world models,' in *The Eleventh International Conference on Learning Representations*, 2023. [Online]. Available: <https://openreview.net/forum?id=vhFu1Acb0xb>.
- [50] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra and M. Riedmiller, 'Playing atari with deep reinforcement learning,' version v1, *arXiv preprint*, Dec. 2013. arXiv: 1312.5602 [cs.LG]. [Online]. Available: <https://arxiv.org/abs/1312.5602>.

- [51] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg and D. Hassabis, 'Human-level control through deep reinforcement learning,' *Nature*, vol. 518, no. 7540, 2015. DOI: 10.1038/nature14236. [Online]. Available: <https://www.nature.com/articles/nature14236>.
- [52] A. Moitra, 'Gaussian mixture models,' Massachusetts Institute of Technology, Lecture Notes Chapter 6, 2015, Unpolished, incomplete course notes. [Online]. Available: https://ocw.mit.edu/courses/18-409-algorithmic-aspects-of-machine-learning-spring-2015/e339520c4069ca5e785b29a3c604470e_MIT18_409S15_chapp6.pdf.
- [53] D. C. Montgomery, E. A. Peck and G. G. Vining, *Introduction to Linear Regression Analysis*, Fifth Edition. John Wiley & Sons, Inc., 2012.
- [54] H. Müller and D. Kudenko, *Improving the effectiveness of potential-based reward shaping in reinforcement learning*, 2025. arXiv: 2502.01307 [cs.LG]. [Online]. Available: <https://arxiv.org/abs/2502.01307>.
- [55] S. Nagendra, N. Podila, R. Ugarakhod and K. George, 'Comparison of reinforcement learning algorithms applied to the cart-pole problem,' *arXiv preprint*, 2018. arXiv: 1810.01940 [cs.LG]. [Online]. Available: <https://arxiv.org/abs/1810.01940>.
- [56] Z. Padamsey and N. L. Rochefort, 'Paying the brain's energy bill,' *Current Opinion in Neurobiology*, vol. 78, p. 102668, 2023, ISSN: 0959-4388. DOI: <https://doi.org/10.1016/j.conb.2022.102668>. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0959438822001623>.
- [57] C. Peters, T. Stewart, R. Westc and B. Esfandiari. 'Dynamic action selection in openai using spiking neural networks.' Accessed: Apr. 4, 2025. (2019), [Online]. Available: <https://www.aaai.org/ocs/index.php/FLAIRS/FLAIRS19/paper/view/18312/17429>.
- [58] S. J. Russell and P. Norvig, *Artificial Intelligence: A modern approach*, 3rd ed. Harlow (England): Pearson, 2016.

- [59] R. S. Sadavarte, R. Raj and B. Sathish Babu, 'Solving the lunar lander problem using reinforcement learning,' in *2021 IEEE International Conference on Computation System and Information Technology for Sustainable Solutions (CSITSS)*, 2021, pp. 1–6. DOI: 10.1109/CSITSS54238.2021.9682970.
- [60] A. Sato, 'Model-free reinforcement learning for financial portfolios: A brief survey,' *arXiv preprint*, 2018. arXiv: 1904.04973 [q-fin.PM]. [Online]. Available: <https://arxiv.org/abs/1904.04973>.
- [61] J. Schrittwieser, I. Antonoglou, T. Hubert, K. Simonyan, L. Sifre, S. Schmitt, A. Guez, E. Lockhart, D. Hassabis, T. Graepel, T. Lillicrap and D. Silver, 'Mastering atari, go, chess and shogi by planning with a learned model,' *Nature*, vol. 588, no. 7839, pp. 604–609, Dec. 2020, ISSN: 1476-4687. DOI: 10.1038/s41586-020-03051-4. [Online]. Available: <http://dx.doi.org/10.1038/s41586-020-03051-4>.
- [62] J. Schulman, F. Wolski, P. Dhariwal, A. Radford and O. Klimov, 'Proximal policy optimization algorithms,' *arXiv preprint*, 2017. arXiv: 1707.06347 [cs.LG]. [Online]. Available: <https://arxiv.org/abs/1707.06347>.
- [63] M. Schwarzer, J. Obando-Ceron, A. Courville, M. Bellemare, R. Agarwal and P. S. Castro, *Bigger, better, faster: Human-level atari with human-level efficiency*, 2023. arXiv: 2305.19452 [cs.LG]. [Online]. Available: <https://arxiv.org/abs/2305.19452>.
- [64] R. Sekar, O. Rybkin, K. Daniilidis, P. Abbeel, D. Hafner and D. Pathak, *Planning to explore via self-supervised world models*, 2020. arXiv: 2005.05960 [cs.LG]. [Online]. Available: <https://arxiv.org/abs/2005.05960>.
- [65] A. Srinivas, M. Laskin and P. Abbeel, *Curl: Contrastive unsupervised representations for reinforcement learning*, 2020. arXiv: 2004.04136 [cs.LG]. [Online]. Available: <https://arxiv.org/abs/2004.04136>.
- [66] D. Steckelmacher, 'Model-free reinforcement learning for real-world robots,' Ph.D. dissertation, Vrije Universiteit Brussel, 2020. [Online]. Available: <https://researchportal.vub.be/en/publications/model-free-reinforcement-learning-for-real-world-robots>.

- [67] Stortinget, *Lov om norges klimamål (climate change act)*, <https://lovdata.no/dokument/NLE/lov/2017-06-16-60>, LOV-2017-06-16-60; accessed 2025-05-14, Jun. 2017.
- [68] E. Strubell, A. Ganesh and A. McCallum, 'Energy and policy considerations for deep learning in NLP,' in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, A. Korhonen, D. Traum and L. Màrquez, Eds., Florence, Italy: Association for Computational Linguistics, Jul. 2019, pp. 3645–3650. DOI: 10.18653/v1/P19-1355. [Online]. Available: <https://aclanthology.org/P19-1355/>.
- [69] M. F. Tandia, *Visualization method: Sne vs t-sne*, <https://www.linkedin.com/pulse/visualization-method-sne-vs-t-sne-implementation-using-tandia/>, 2020.
- [70] M. Towers, A. Kwiatkowski, J. Terry, J. U. Balis, G. D. Cola, T. Deleu, M. Goulão, A. Kallinteris, M. Krimmel, A. KG, R. Perez-Vicente, A. Pierré, S. Schulhoff, J. J. Tai, H. Tan and O. G. Younis, 'Gymnasium: A standard interface for reinforcement learning environments,' *arXiv preprint*, 2024. arXiv: 2407.17032 [cs.LG]. [Online]. Available: <https://arxiv.org/abs/2407.17032>.
- [71] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser and I. Polosukhin, 'Attention is all you need,' in *Advances in Neural Information Processing Systems*, I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan and R. Garnett, Eds., vol. 30, Curran Associates, Inc., 2017. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf.
- [72] O. Vinyals, I. Babuschkin, W. M. Czarnecki, M. Mathieu, A. Dudzik, J. Chung and D. Silver, 'Grandmaster level in starcraft ii using multi-agent reinforcement learning,' *Nature*, vol. 575, no. 7782, 2019. DOI: 10.1038/s41586-019-1724-z. [Online]. Available: <https://www.nature.com/articles/s41586-019-1724-z>.
- [73] S. Wang, S. Liu, W. Ye, J. You and Y. Gao, *Efficientzero v2: Mastering discrete and continuous control with limited data*, 2024. arXiv: 2403.00564 [cs.LG]. [Online]. Available: <https://arxiv.org/abs/2403.00564>.

- [74] Z. Wang, J. Wang, Q. Zhou, B. Li and H. Li, *Sample-efficient reinforcement learning via conservative model-based actor-critic*, 2021. arXiv: 2112.10504 [cs.LG]. [Online]. Available: <https://arxiv.org/abs/2112.10504>.
- [75] Z. Wang, J. Zhang and Y. Wang, 'Model-free reinforcement learning for asset allocation,' *arXiv preprint*, 2022. arXiv: 2209.10458 [q-fin.PM]. [Online]. Available: <https://arxiv.org/abs/2209.10458>.
- [76] M. Wattenberg, F. Viégas and I. Johnson, 'How to use t-sne effectively,' *Distill*, vol. 1, Oct. 2016. DOI: 10.23915/distill.00002.
- [77] A. Wong, J. de Nobel, T. Bäck, A. Plaat and A. V. Kononova, *Solving deep reinforcement learning tasks with evolution strategies and linear policy networks*, 2024. arXiv: 2402.06912 [cs.LG]. [Online]. Available: <https://arxiv.org/abs/2402.06912>.
- [78] J. Wu, Z. Huang and C. Lv, *Uncertainty-aware model-based reinforcement learning with application to autonomous driving*, 2021. arXiv: 2106.12194 [cs.R0]. [Online]. Available: <https://arxiv.org/abs/2106.12194>.
- [79] J. Xie, R. B. Girshick and A. Farhadi, 'Unsupervised deep embedding for clustering analysis,' *CoRR*, vol. abs/1511.06335, 2015. arXiv: 1511.06335. [Online]. Available: <http://arxiv.org/abs/1511.06335>.
- [80] J. Xu. 'How to beat the cartpole game in 5 lines.' Published in TDS Archive, Medium, February 21, 2021. Accessed: April 1, 2025. (2021), [Online]. Available: <https://medium.com/data-science/how-to-beat-the-cartpole-game-in-5-lines-5ab4e738c93f>.
- [81] H. Yang, 'An independent study of reinforcement learning and autonomous driving,' *arXiv preprint*, 2021. arXiv: 2110.07729 [cs.AI]. [Online]. Available: <https://arxiv.org/abs/2110.07729>.
- [82] D. Yarats, R. Fergus, A. Lazaric and L. Pinto, *Mastering visual continuous control: Improved data-augmented reinforcement learning*, 2021. arXiv: 2107.09645 [cs.AI]. [Online]. Available: <https://arxiv.org/abs/2107.09645>.

- [83] W. Ye, S. Liu, T. Kurutach, P. Abbeel and Y. Gao, *Mastering atari games with limited data*, 2021. arXiv: 2111.00210 [cs.LG]. [Online]. Available: <https://arxiv.org/abs/2111.00210>.

A. Hyperparameters

Parameter	Value
Exploration rate	0.1
Gaussian width (σ)	{0.1, 0.3, 0.5, 1.0, 2.5, 5.0}
Discount factor (γ)	1.0
Transition model	Direct transition model

Table A.1: Hyperparameters used for experiment I, Figure 5.1.

Parameter	Value
Exploration rate	0.1
Gaussian width (σ)	0.5
Discount factor (γ)	1.0
k	{100, 250, 500, 750, 1000, 1500, 2000, 2500, 3000, 3500, 4000, 4500, 5000}
Transition model	Direct transition model

Table A.2: Hyperparameters used for experiment II, Figure 5.3.

Parameter	Value
Exploration rate	0.1
Gaussian width (σ)	0.5
Discount factor (γ)	1.0
k	3000
Transition model	Direct transition model

Table A.3: Hyperparameters used in experiment III, Figure 5.6.

Parameter	Value
Exploration rate	0.1
Gaussian width (σ)	0.5
Discount factor (γ)	1.0
k	3000
Transition model	Direct transition model
Temperature	{0.01, 0.05, 0.1, 0.5, 1, 5, 10, 25, 50, 100}

Table A.4: Hyperparameters used in experiment III, Figure 5.7.

Parameter	Value
Exploration rate	0.1
Gaussian width (σ)	0.5
Discount factor (γ)	1.0
k	3000
Transition model	Direct transition model
Upper bound	{3, 15}

Table A.5: Hyperparameters used in experiment III, Figure 5.9.

Parameter	Value
Exploration rate	0.1
Gaussian width (σ) policy	0.3
Gaussian width (σ) clustering	0.2
Discount factor (γ)	1.0
k	4000
Transition model	Direct transition model
Temperature	{1, 50}

Table A.6: Hyperparameters used in experiment IV, Figure 5.11.

Parameter	Value
Exploration rate	0.1
Gaussian width (σ)	0.5
Discount factor (γ)	1.0
k	4500
Transition model	Direct transition model
Covariance type	Full
Random state	42
Init params	k-means

Table A.7: Hyperparameters used in experiment IV, Figure 5.12.

Parameter	Value
Exploration rate	0.1
Gaussian width (σ)	{0.1, 0.3, 0.5, 1.0, 2.5, 5.0}
Discount factor (γ)	1.0
Transition model	Delta transition model
Error threshold (ϵ)	0.01

Table A.8: Hyperparameters for experiment V, Figure 5.13.

Parameter	Value
Exploration rate	0.1
Gaussian width (σ)	0.5
Discount factor (γ)	1.0
k	3000
Transition model	Delta transition model
Error threshold (ϵ)	$\{10^{-2}, 10^{-3}, 10^{-4}, 10^{-5}\}$

Table A.9: Hyperparameters for experiment V, Figure 5.14.

Parameter	Value
Exploration rate	0.1
Gaussian width (σ)	0.5
Discount factor (γ)	1.0
k	$\{100, 250, 500, 750, 1000, 1500, 2000, 2500, 3000, 3500, 4000, 4500, 5000\}$
Transition model	Delta transition model
Error threshold (ϵ)	10^{-2}

Table A.10: Hyperparameters for experiment V, Figure 5.15.

Parameter	Value
Exploration rate	0.1
Gaussian width (σ)	$\{0.1, 0.3, 0.5, 1.0, 2.5, 5.0\}$
Discount factor (γ)	1.0
Transition model	Delta transition model
Error threshold (ϵ)	10^{-2}

Table A.11: Hyperparameters for experiment VI, Figure 5.16.

Parameter	Value
Exploration rate	0.1
Gaussian width (σ)	0.5
Discount factor (γ)	1.0
k	$\{100, 250, 500, 750, 1000, 1500, 2000, 2500, 3000, 3500, 4000, 4500, 5000\}$
Transition model	Delta transition model
Error threshold (ϵ)	10^{-2}

Table A.12: Hyperparameters for experiment IV, Figure 5.17.

Parameter	Value
Exploration rate	0.1
Gaussian width (σ)	1.0
Discount factor (γ)	1.0
k	{100, 500, 1000, 3000, 4500}
Transition model	Delta transition model
Error threshold (ϵ)	10^{-2}

Table A.13: Hyperparameters for experiment VII, Figure 5.18.

Parameter	Value
Exploration rate	0.1
Gaussian width (σ)	2.5
Discount factor (γ)	1.0
k	{100, 500, 1000, 3000, 4500}
Transition model	Delta transition model
Error threshold (ϵ)	10^{-2}

Table A.14: Hyperparameters for experiment VII, Figure 5.19.

Parameter	Value
Exploration rate	0.1
Gaussian width (σ)	5.0
Discount factor (γ)	1.0
k	{100, 500, 1000, 3000, 4500}
Transition model	Delta transition model
Error threshold (ϵ)	10^{-2}

Table A.15: Hyperparameters for experiment VII, Figure 5.20.

Parameter	Value
Exploration rate	0.1
Gaussian width (σ)	1.0
Discount factor (γ)	1.0
k	{100, 500, 1000, 3000, 4500}
Transition model	Direct transition model

Table A.16: Hyperparameters for experiment VII, Figure 5.21.

Parameter	Value
Exploration rate	0.1
Gaussian width (σ)	2.5
Discount factor (γ)	1.0
k	{100, 500, 1000, 3000, 4500}
Transition model	Direct transition model

Table A.17: Hyperparameters for experiment VII, Figure 5.22.

Parameter	Value
Exploration rate	0.1
Gaussian width (σ)	5.0
Discount factor (γ)	1.0
k	{100, 500, 1000, 3000, 4500}
Transition model	Direct transition model

Table A.18: Hyperparameters for experiment VII, Figure 5.23.

Parameter	Value
Exploration rate	0.1
Gaussian width (σ) encoding	0.5
Gaussian width (σ) policy	0.5
Discount factor (γ)	1.0
k	3000
Transition model	Delta transition model
Error threshold (ϵ)	10^{-2}
Temperature	50
New dimensions (n) per existing dimension	10

Table A.19: Hyperparameters for experiment VIII, Figure 5.24.

Parameter	Value
Exploration rate	0.1
Gaussian width (σ)	0.5
Discount factor (γ)	1.0
k	3000
Transition model	Delta transition model
Error threshold (ϵ)	10^{-2}
New dimensions (n) per existing dimension	4

Table A.20: Hyperparameters for experiment VIII, Figure 5.25.

Parameter	Value
Exploration rate	0.1
Gaussian width (σ)	200.0
Discount factor (γ)	1.0
k	500
Transition model	Delta transition model
Error threshold (ϵ)	10^{-2}
New dimensions (n) per existing dimension	4

Table A.21: Hyperparameters for experiment VIII, Figure 5.26.

Parameter	Value
Learning rate (α)	1×10^{-3}
Initial exploration rate (ϵ_{start})	0.1
Replay buffer capacity	5,000
Batch size	64
Target-network update freq. (steps)	100

Table A.22: Hyperparameters for the DQN agent experiment IX (training), 5.27.

Parameter	Value
Learning rate (α)	1×10^{-3}
Initial exploration rate (ϵ_{start})	0
Replay buffer capacity	5,000
Batch size	64
Target-network update freq. (steps)	100

Table A.23: Hyperparameters for the DQN agent experiment IX (testing), 5.28.

Parameter	Value
Learning rate (α)	1.0
Initial exploration rate (ϵ)	0.1
State bins per dimension	10

Table A.24: Hyperparameters for the Q-learning baseline (training), ??.

Parameter	Value
Learning rate (α)	1.0
Minimum exploration rate (ϵ_{min})	0
State bins per dimension	10

Table A.25: Hyperparameters for the Q-learning baseline (testing), ??.

B. Q-Learning Reproduction Attempt

This appendix presents the results of an attempted reproduction of the Q-learning experiment reported by Kumar [45], which claimed to solve CartPole-v0 within 200 episodes. The implementation used in this reproduction matched the original as closely as possible, including the use of identical hyperparameters and a similar code structure. To ensure robustness and avoid cherry-picking or random variance, the results shown in Figure B.1 represent the mean episode reward over 100 independent runs.

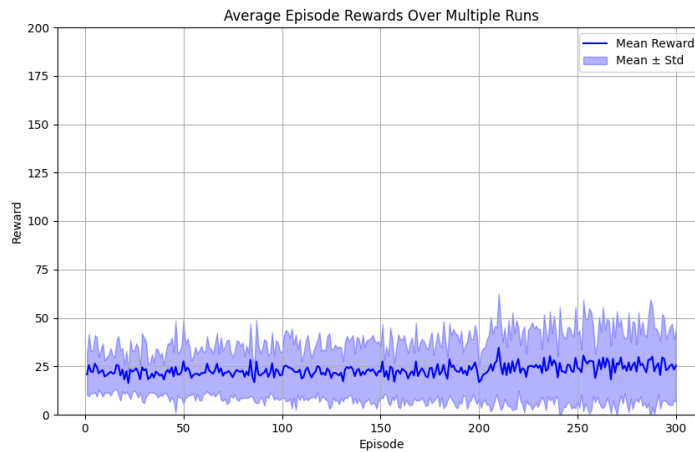


Figure B.1: Mean episode reward over 100 runs in the Q-learning reproduction attempt.

The hyperparameters used in this experiment are listed in Table B.1. Both the learning rate (α) and the exploration rate (ϵ) were initialised at 1.0 and decayed to a minimum of 0.1, consistent with the configuration reported by Kumar [45].

Parameter	Value
Learning rate (α)	1.0
Minimum learning rate	0.1
Discount factor (γ)	0.99
Initial exploration rate (ϵ)	1.0
Minimum ϵ	0.10
ϵ -decay rate	0.99

Table B.1: Hyperparameters used in the Q-learning reproduction attempt

The discrepancy in performance suggests either a missing implementation detail in the original study, the presence of undocumented modifications or hyperparameter tuning, or the use of additional techniques such as reward shaping.

C. Raw data

The raw data for all the experiments can be found in the attached materials or at <https://github.com/SveinKare/BachelorThesis-logs>.